

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное

образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет «Систем управления и робототехники»

Лабораторная работа 4

по дисциплине

«Теория автоматического управления»

Вариант 15

Выполнил:

Скалиуш Артём (409567)

Преподаватель:

Пашенко Артем Витальевич



Санкт-Петербург 2026

Содержание

Задание 1. Синтез регулятора по уравнениям Франкиса-Дэвисона	3
1) Собственные числа матрицы Γ_f и характер возмущения	4
2) Построение генератора задающего воздействия	4
3) Синтез стабилизирующей компоненты K	4
4) Синтез следящей компоненты K_g	6
5) Синтез компенсирующей компоненты K_f	7
6) Моделирование	8
а) Разомкнутая система $u = 0$	9
б) Только feedback-компонента $u = Kx$	11
в) Без следящей компоненты $u = Kx + K_f w_f$	13
г) Без компенсирующей компоненты $u = Kx + K_g w_g$	15
д) Полный регулятор $u = Kx + K_g w_g + K_f w_f$	17
Вывод по заданию 1	19
Задание 2. Слежение и компенсация по выходу	19
1) Наблюдатель задающего воздействия	19
2) Формирование расширенной системы	20
3) Синтез наблюдателя расширенной размерности	21
4) Моделирование замкнутой системы	22
Вывод по заданию 2	28
Задание 3. Наблюдатели возмущения	28
1) Наблюдатель возмущения по состоянию	29
2) Наблюдатель возмущения по выходу	30
3) Моделирование и сравнение	30
а) Наблюдатель возмущения по состоянию	31
б) Наблюдатель возмущения по выходу	35
Вывод по заданию 3	38
Общий вывод	38
Приложение	40
Код для задания 1	40
Код для задания 2	48
Код для задания 3	58

Задание 1. Синтез регулятора по уравнениям Франкиса-Дэвисона

В этом задании требуется:

- определить модели задающего воздействия и внешнего возмущения;
- синтезировать стабилизирующую, следящую и компенсирующую компоненты регулятора;
- проверить условия существования решений уравнений Франкиса-Дэвисона;
- сравнить работу системы для нескольких структур закона управления.

Для варианта 15 в Задании 1 используются объект №10 из Таблицы 2 и генератор №15 из Таблицы 3. Тогда

$$A = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix},$$
$$B_f = \begin{bmatrix} -2 & 1 \\ 0 & 0 \\ 2 & 0 \end{bmatrix}, \quad C = [2 \quad -2 \quad 1], \quad D = 10, \quad D_f = [0 \quad 7].$$

Объект управления имеет вид

$$\dot{x} = Ax + Bu + B_f f, \quad x(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$
$$y = Cx + Du + D_f f.$$

Генератор внешнего возмущения задается формулами

$$\dot{w}_f = \Gamma_f w_f, \quad w_f(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad f = Y_f w_f,$$

где

$$\Gamma_f = \begin{bmatrix} -40 & 16 & 9 & 7 \\ -64 & 25 & 14 & 12 \\ -26 & 11 & 7 & 3 \\ -48 & 18 & 14 & 8 \end{bmatrix},$$
$$Y_f = \begin{bmatrix} -12 & 2 & 2 & 6 \\ 8 & -8 & 12 & -3 \end{bmatrix}.$$

Задающее воздействие из условия равно

$$g(t) = 19 \sin t - 10.$$

Требуется построить регулятор полной информации

$$u = Kx + K_g w_g + K_f w_f,$$

обеспечивающий выполнение условия

$$\lim_{t \rightarrow \infty} |g(t) - y(t)| = 0.$$

1) Собственные числа матрицы Γ_f и характер возмущения

Собственные числа матрицы Γ_f равны

$$\sigma(\Gamma_f) = \{3i, -3i, 2i, -2i\}.$$

Следовательно, генератор внешнего возмущения содержит две гармонические моды с частотами $\omega_1 = 3$ и $\omega_2 = 2$. Это означает, что компоненты вектора $f(t)$ являются линейными комбинациями синусоид и косинусоид с этими частотами, то есть внешнее возмущение является ограниченным периодическим двухчастотным сигналом.

2) Построение генератора задающего воздействия

Чтобы породить сигнал $g(t) = 19 \sin t - 10$, генератор должен содержать:

- колебательную моду с частотой 1;
- постоянную моду с собственным числом 0.

Выберем вещественный генератор вида

$$\Gamma_g = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad Y_g = [19 \ 0 \ -10], \quad w_g(0) = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

Тогда система

$$\dot{w}_g = \Gamma_g w_g$$

имеет решение

$$w_{g,1}(t) = \sin t, \quad w_{g,2}(t) = \cos t, \quad w_{g,3}(t) = 1.$$

Следовательно,

$$g(t) = Y_g w_g(t) = 19w_{g,1}(t) - 10w_{g,3}(t) = 19 \sin t - 10,$$

то есть выбранные матрицы Γ_g , Y_g и начальное условие $w_g(0)$ действительно порождают требуемое задающее воздействие. Собственные числа матрицы Γ_g равны

$$\sigma(\Gamma_g) = \{i, -i, 0\}.$$

3) Синтез стабилизирующей компоненты K

Сначала проверим объект на управляемость и стабилизируемость. Вычислим матрицу управляемости Калмана.

$$AB = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix} \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix} = \begin{bmatrix} 18 \\ 8 \\ -8 \end{bmatrix}.$$

$$A^2B = A(AB) = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix} \begin{bmatrix} 18 \\ 8 \\ -8 \end{bmatrix} = \begin{bmatrix} 32 \\ 20 \\ -20 \end{bmatrix}.$$

$$U = [B, AB, A^2B] = \begin{bmatrix} 2 & 18 & 32 \\ -2 & 8 & 20 \\ 2 & -8 & -20 \end{bmatrix}.$$

$$\det(U) = 0, \quad \text{rank}(U) = 2 < 3.$$

Следовательно, объект не полностью управляем.

Найдем собственные числа матрицы A :

$$\sigma(A) = \{1 + i, 1 - i, -2\}.$$

Проверим их по критерию Хаутуса.

Для $\lambda = 1 + i$:

$$H(1 + i) = [(1 + i)I - A, B] = \begin{bmatrix} -3 + i & -1 & -6 & 2 \\ -2 & 1 + i & -2 & -2 \\ 2 & 2 & 5 + i & 2 \end{bmatrix}.$$

Для $\lambda = 1 - i$:

$$H(1 - i) = [(1 - i)I - A, B] = \begin{bmatrix} -3 - i & -1 & -6 & 2 \\ -2 & 1 - i & -2 & -2 \\ 2 & 2 & 5 - i & 2 \end{bmatrix}.$$

Для $\lambda = -2$:

$$H(-2) = [-2I - A, B] = \begin{bmatrix} -6 & -1 & -6 & 2 \\ -2 & -2 & -2 & -2 \\ 2 & 2 & 2 & 2 \end{bmatrix}.$$

Получаем

$$\text{rank}(H(1 + i)) = 3, \quad \text{rank}(H(1 - i)) = 3, \quad \text{rank}(H(-2)) = 2.$$

Следовательно, неуправляемой является только мода $\lambda = -2$, причем она уже устойчива. Значит объект стабилизируем.

Стабилизирующую компоненту K найдем методом LQR при

$$Q = I, \quad R = 1.$$

В результате синтеза получаем следующую матрицу регулятора

$$K = [-2.963973 \quad -0.547344 \quad -2.331086].$$

Тогда

$$A + BK = \begin{bmatrix} -1.927945 & -0.094689 & 1.337829 \\ 7.927945 & 1.094689 & 6.662171 \\ -7.927945 & -3.094689 & -8.662171 \end{bmatrix}$$

и

$$\sigma(A + BK) = \{-3.747714 + 2.836435i, -3.747714 - 2.836435i, -2\}.$$

Все собственные числа находятся в левой полуплоскости, следовательно, матрица $A + BK$ устойчива.

4) Синтез следящей компоненты K_g

Для регулятора полной информации при уже найденной матрице K используем матричные уравнения Франкиса-Дэвисона:

$$P_g \Gamma_g - (A + BK)P_g = BK_g,$$

$$(C + DK)P_g + DK_g = Y_g.$$

Сначала проверим условие существования решения. Для каждого собственного числа матрицы Γ_g должна выполняться проверка ранга матрицы

$$R(\lambda) = \begin{bmatrix} A - \lambda I & B \\ C & D \end{bmatrix}.$$

Так как матрица $R(\lambda)$ квадратная размера 4×4 , достаточно показать, что ее определитель не равен нулю.

Для $\lambda = 0$:

$$R(0) = \begin{bmatrix} 4 & 1 & 6 & 2 \\ 2 & 0 & 2 & -2 \\ -2 & -2 & -4 & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix}, \quad \det R(0) = -24 \neq 0.$$

Для $\lambda = i$:

$$R(i) = \begin{bmatrix} 4 - i & 1 & 6 & 2 \\ 2 & -i & 2 & -2 \\ -2 & -2 & -4 - i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(i) = -14 + 18i \neq 0.$$

Для $\lambda = -i$:

$$R(-i) = \begin{bmatrix} 4 + i & 1 & 6 & 2 \\ 2 & i & 2 & -2 \\ -2 & -2 & -4 + i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(-i) = -14 - 18i \neq 0.$$

Следовательно, для всех собственных чисел матрицы Γ_g условие существования решения выполнено.

Неизвестными являются матрицы $P_g \in R^{3 \times 3}$ и $K_g \in R^{1 \times 3}$. После векторизации получаем линейную систему из 12 скалярных уравнений относительно 12 неизвестных. Ее решение равно

$$P_g = \begin{bmatrix} 1.461538 & 26.307692 & -11.666667 \\ 8.038462 & 21.192308 & -10.0 \\ -8.038462 & -21.192308 & 10.0 \end{bmatrix},$$

$$K_g = [-5.987345 \quad 41.269841 \quad -18.408935].$$

Проверка подстановкой дает невязки порядка машинного нуля:

$$\max |P_g \Gamma_g - (A + BK)P_g - BK_g| = 5.68 \cdot 10^{-14},$$

$$\max |(C + DK)P_g + DK_g - Y_g| = 2.84 \cdot 10^{-14}.$$

5) Синтез компенсирующей компоненты K_f

Для общей постановки с возмущением и на входе, и на выходе используются уравнения Франкиса-Дэвисона

$$P_f \Gamma_f - (A + BK)P_f - B_f Y_f = BK_f,$$

$$(C + DK)P_f + DK_f = -D_f Y_f.$$

Проверим условие существования решения для всех собственных чисел матрицы Γ_f .

Для $\lambda = 2i$:

$$R(2i) = \begin{bmatrix} 4 - 2i & 1 & 6 & 2 \\ 2 & -2i & 2 & -2 \\ -2 & -2 & -4 - 2i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(2i) = 16 + 96i \neq 0.$$

Для $\lambda = -2i$:

$$R(-2i) = \begin{bmatrix} 4 + 2i & 1 & 6 & 2 \\ 2 & 2i & 2 & -2 \\ -2 & -2 & -4 + 2i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(-2i) = 16 - 96i \neq 0.$$

Для $\lambda = 3i$:

$$R(3i) = \begin{bmatrix} 4 - 3i & 1 & 6 & 2 \\ 2 & -3i & 2 & -2 \\ -2 & -2 & -4 - 3i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(3i) = 66 + 294i \neq 0.$$

Для $\lambda = -3i$:

$$R(-3i) = \begin{bmatrix} 4+3i & 1 & 6 & 2 \\ 2 & 3i & 2 & -2 \\ -2 & -2 & -4+3i & 2 \\ 2 & -2 & 1 & 10 \end{bmatrix},$$

$$\det R(-3i) = 66 - 294i \neq 0.$$

Следовательно, уравнения компенсации также имеют решение.

Здесь неизвестными являются матрицы $P_f \in R^{3 \times 4}$ и $K_f \in R^{1 \times 4}$. После векторизации получается линейная система из 16 скалярных уравнений относительно 16 неизвестных. Ее решение равно

$$P_f = \begin{bmatrix} 21.409892 & -19.493424 & 14.930550 & -0.650892 \\ 7.163770 & -14.645927 & 3.926185 & 10.769467 \\ -12.702231 & 17.107465 & -6.080031 & -8.000236 \end{bmatrix},$$

$$K_f = [30.590395 \quad -21.056621 \quad 22.236775 \quad -9.499760].$$

Невязки при подстановке также малы:

$$\max |P_f \Gamma_f - (A + BK)P_f - BK_f - B_f Y_f| = 1.07 \cdot 10^{-13},$$

$$\max |(C + DK)P_f + DK_f + D_f Y_f| = 2.27 \cdot 10^{-13}.$$

Следовательно, регулятор

$$u = Kx + K_g w_g + K_f w_f$$

с синтезированными матрицами K , K_g и K_f удовлетворяет уравнениям Франкиса-Дэвисона.

6) Моделирование

Проведем моделирование на интервале $t \in [0, 12]$ для пяти режимов:

1. разомкнутая система $u = 0$;
2. система только с feedback-компонентой $u = Kx$;
3. система без следящей компоненты $u = Kx + K_f w_f$;
4. система без компенсирующей компоненты $u = Kx + K_g w_g$;
5. полная система $u = Kx + K_g w_g + K_f w_f$.

Для краткого сравнения результатов сведем характерные значения в таблицу.

Режим	$\max u $	$\max x $	$\max y $	$\max e $	$e(12)$
$u = 0$	0.000	352520.940	773979.369	773999.564	-773999.564
$u = Kx$	2.623	4.015	63.0	73.0	-16.337
$u = Kx + K_f w_f$	22.271	17.024	285.708	295.708	-20.195
$u = Kx + K_g w_g$	22.861	38.973	291.609	301.609	3.858
$u = Kx + K_g w_g + K_f w_f$	45.132	33.751	514.317	524.317	$-8.60 \cdot 10^{-7}$

Таблица 1. Сравнение режимов моделирования

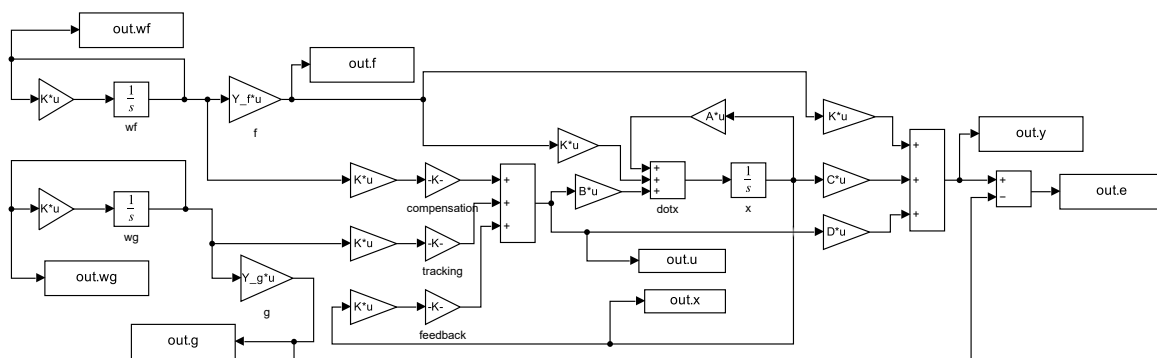


Рисунок 1. Структурная схема моделирования регулятора полной информации

а) Разомкнутая система $u = 0$

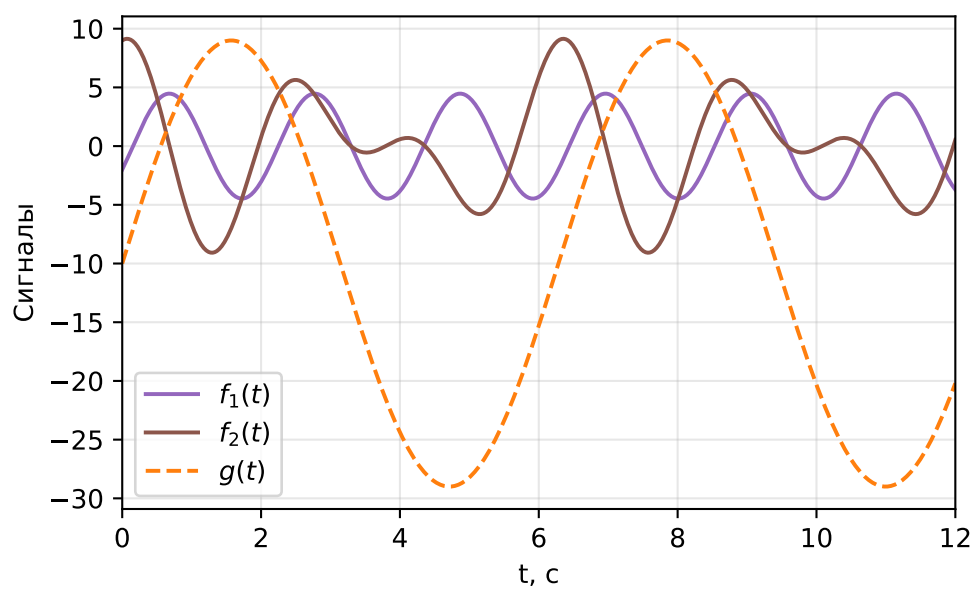


Рисунок 2. Внешнее возмущение и задающее воздействие для разомкнутой системы

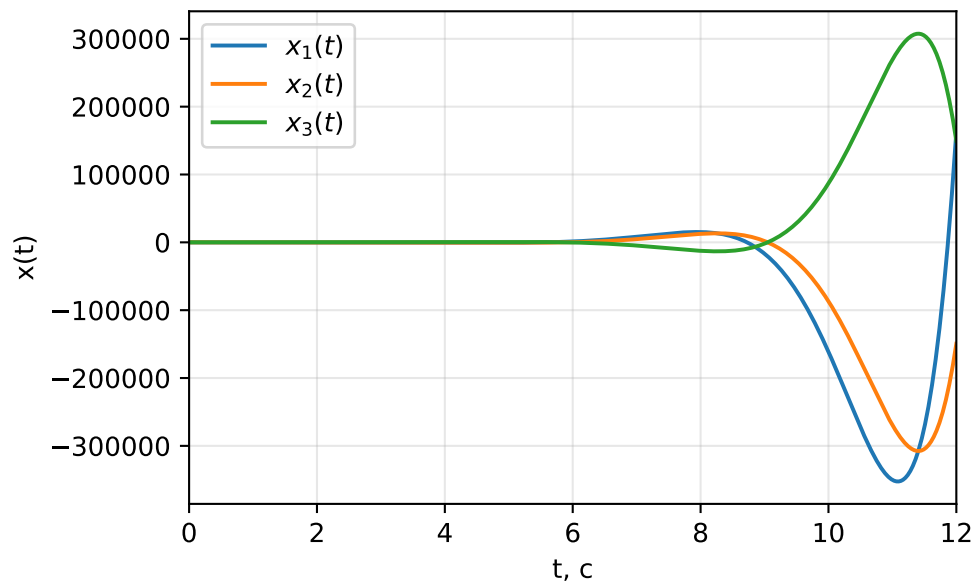


Рисунок 3. Состояние разомкнутой системы

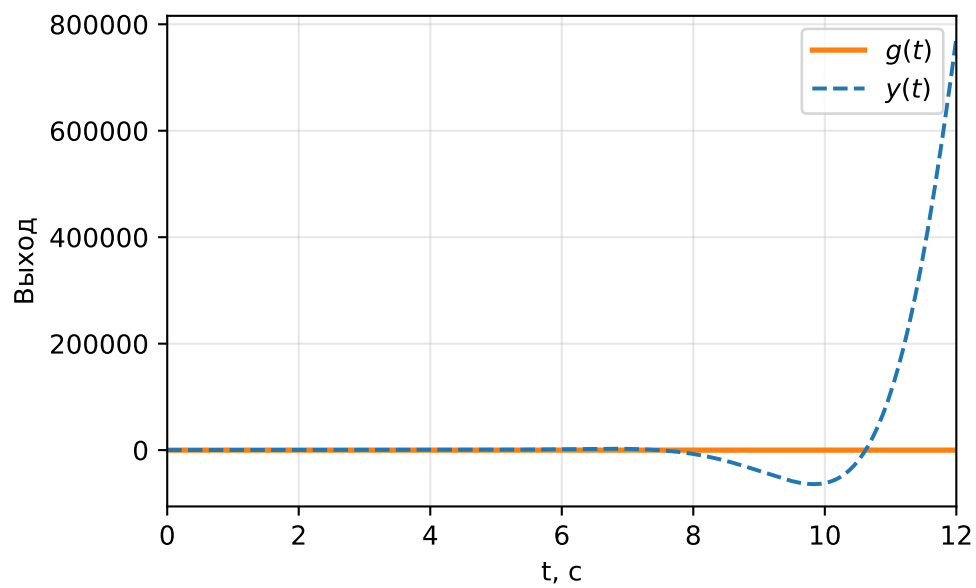


Рисунок 4. Сравнение выхода и задающего воздействия для разомкнутой системы

Без управления состояние объекта быстро нарастает, а выход не следует за заданием. Ошибка регулирования остается большой и не имеет тенденции к затуханию.

б) Только feedback-компонента $u = Kx$

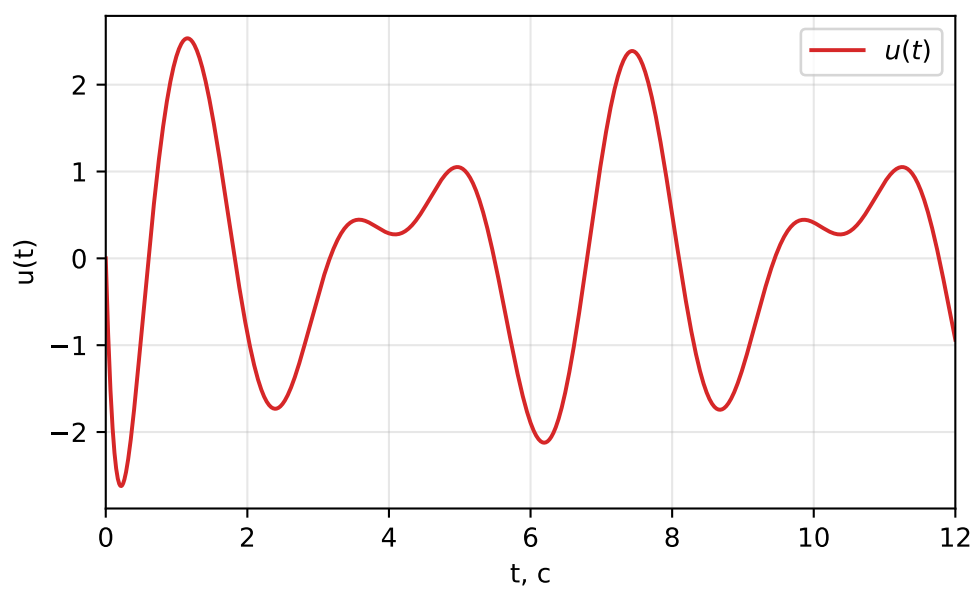


Рисунок 5. Управление при регуляторе $u = Kx$

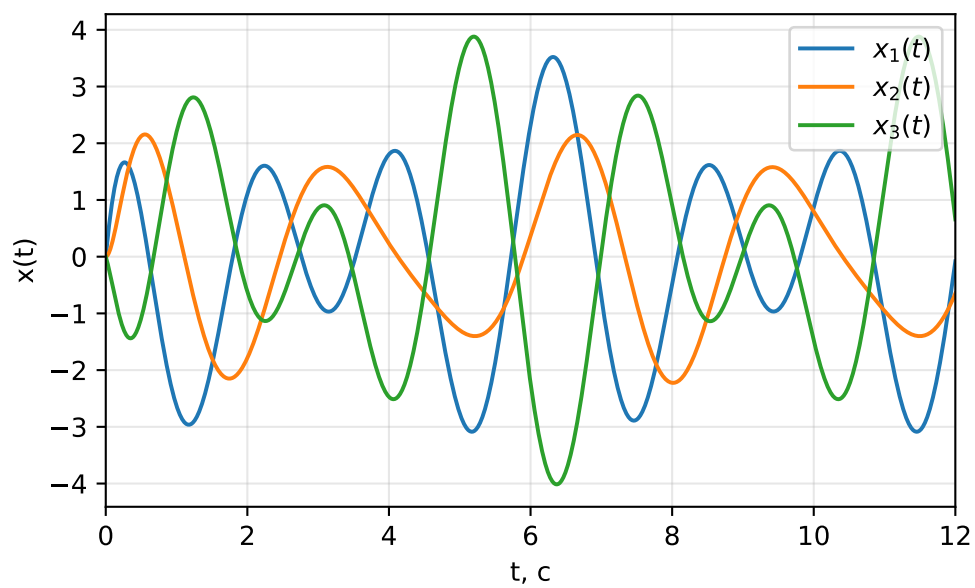


Рисунок 6. Состояние системы при регуляторе $u = Kx$

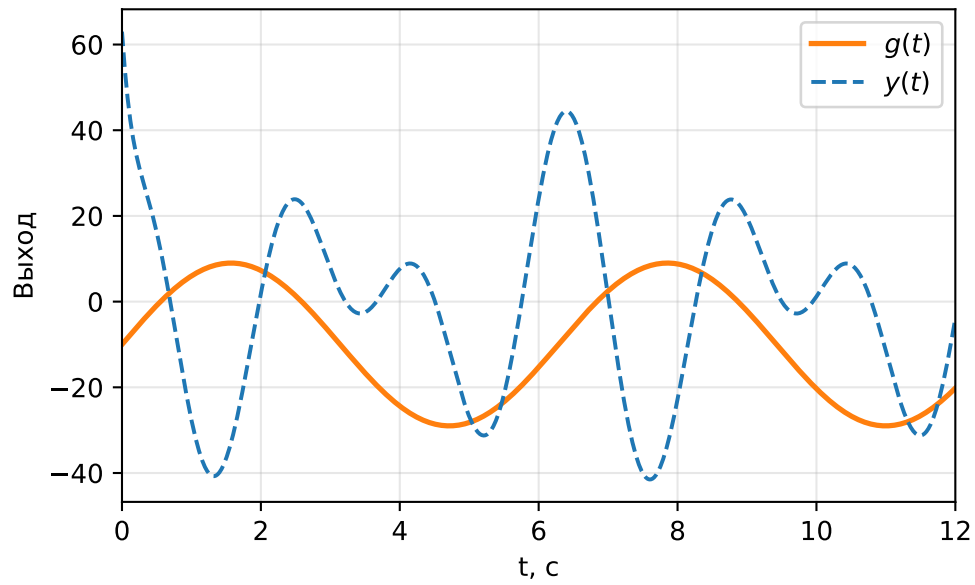


Рисунок 7. Сравнение выхода и задающего воздействия при регуляторе $u = Kx$

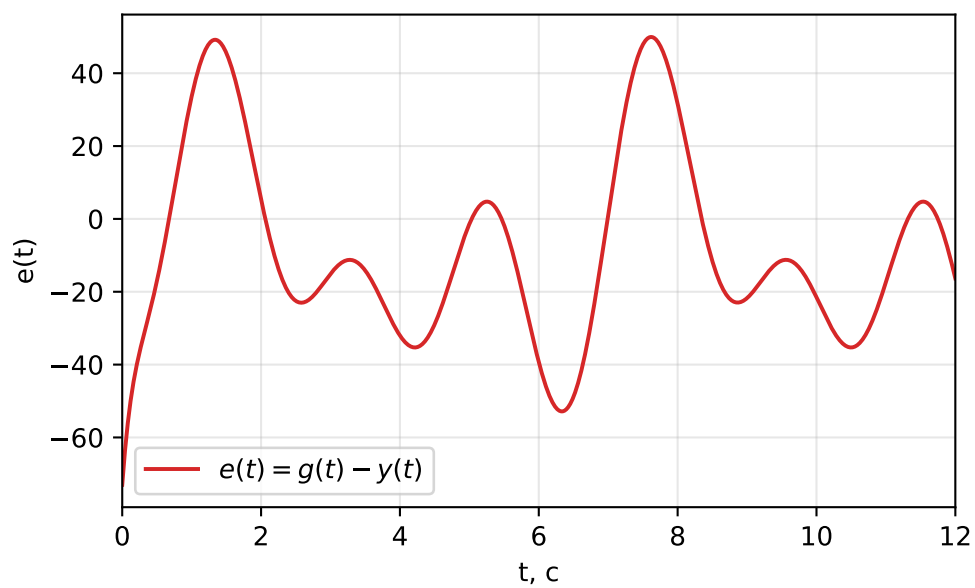


Рисунок 8. Ошибка регулирования при регуляторе $u = Kx$

Одна только стабилизирующая обратная связь ограничивает состояние, однако не решает задачу слежения. Выход остается далеким от задающего воздействия, поскольку генераторы w_g и w_f в закон управления не входят.

в) Без следящей компоненты $u = Kx + K_f w_f$

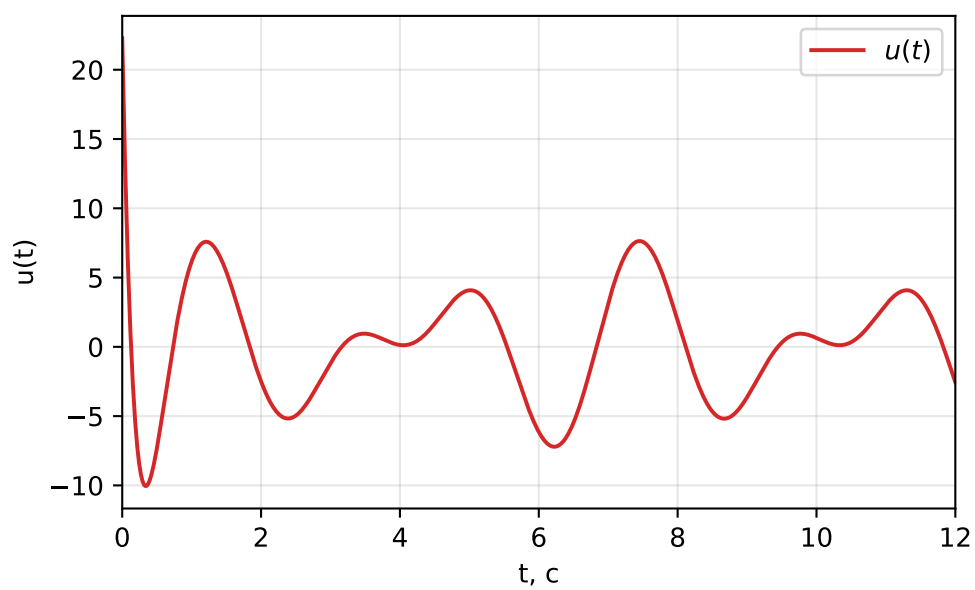


Рисунок 9. Управление без следящей компоненты

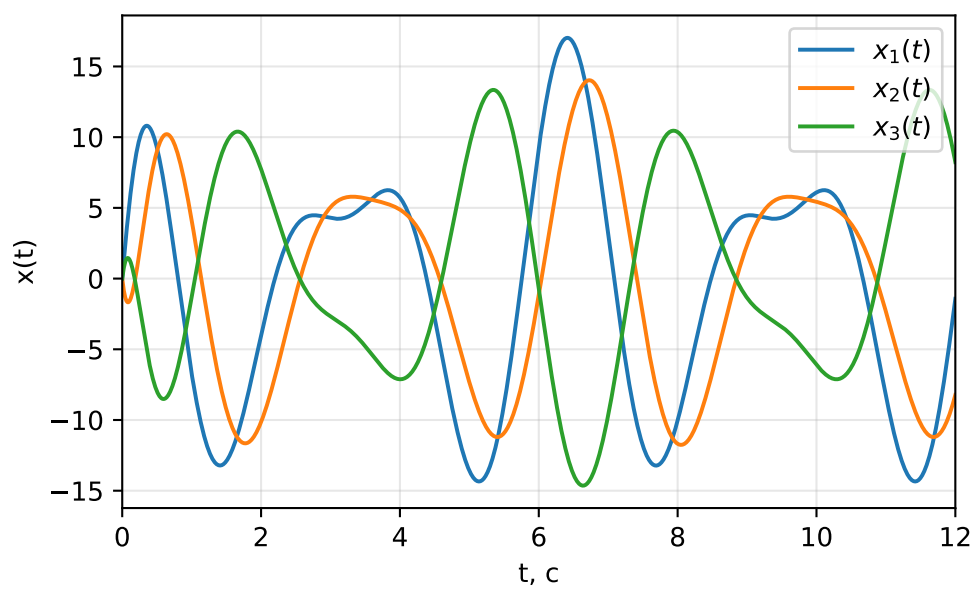


Рисунок 10. Состояние системы без следящей компоненты

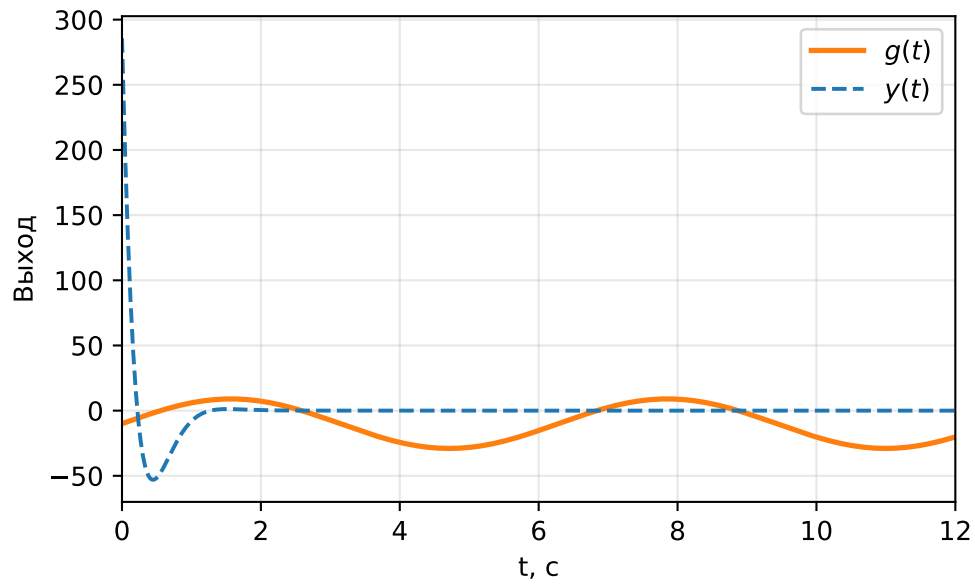


Рисунок 11. Сравнение выхода и задающего воздействия без следящей компоненты

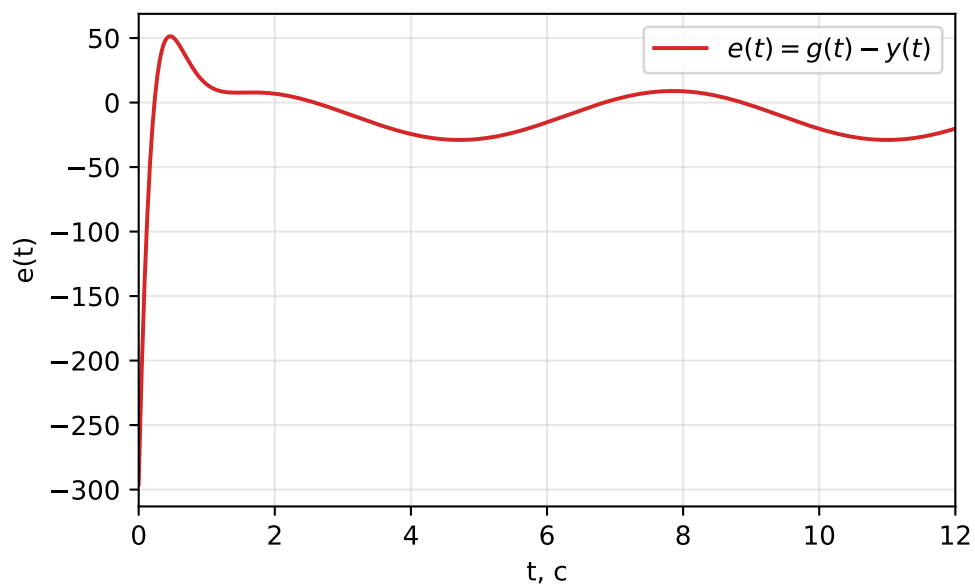


Рисунок 12. Ошибка регулирования без следящей компоненты

Добавление $K_f w_f$ компенсирует известное внешнее возмущение, однако при отсутствии слагаемого $K_g w_g$ система не имеет внутренней модели задающего сигнала. Поэтому ошибка уменьшается по сравнению со случаем только Kx , но к нулю не стремится.

г) Без компенсирующей компоненты $u = Kx + K_g w_g$

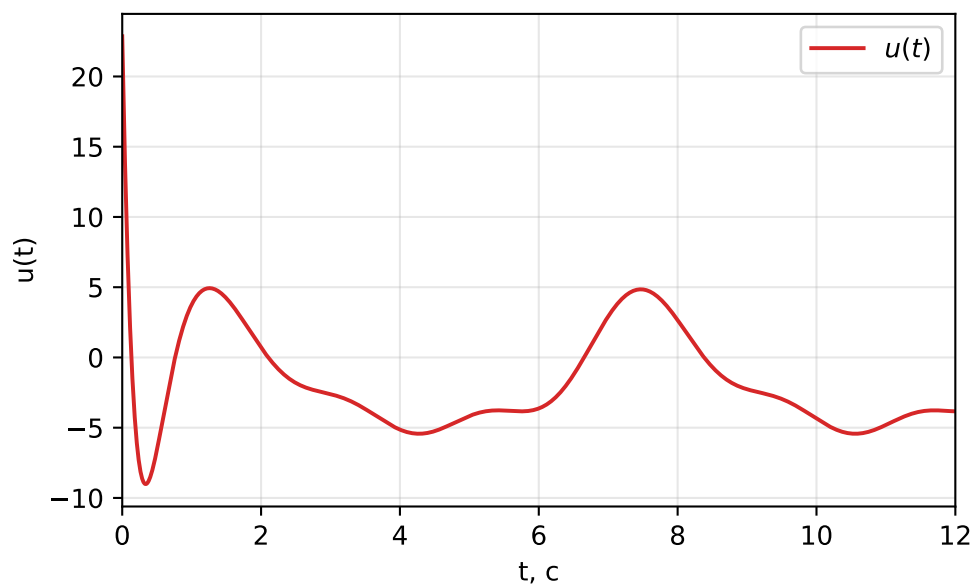


Рисунок 13. Управление без компенсирующей компоненты

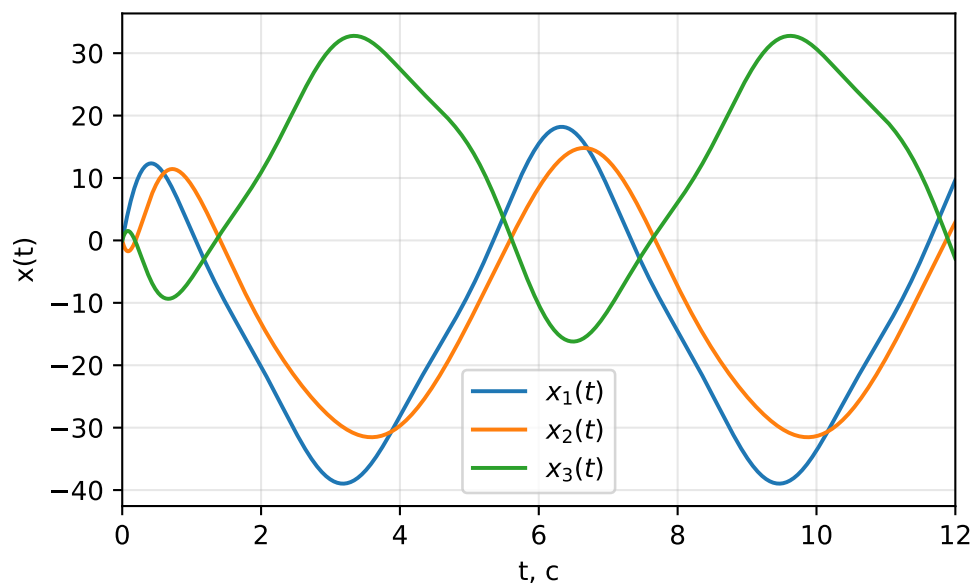


Рисунок 14. Состояние системы без компенсирующей компоненты

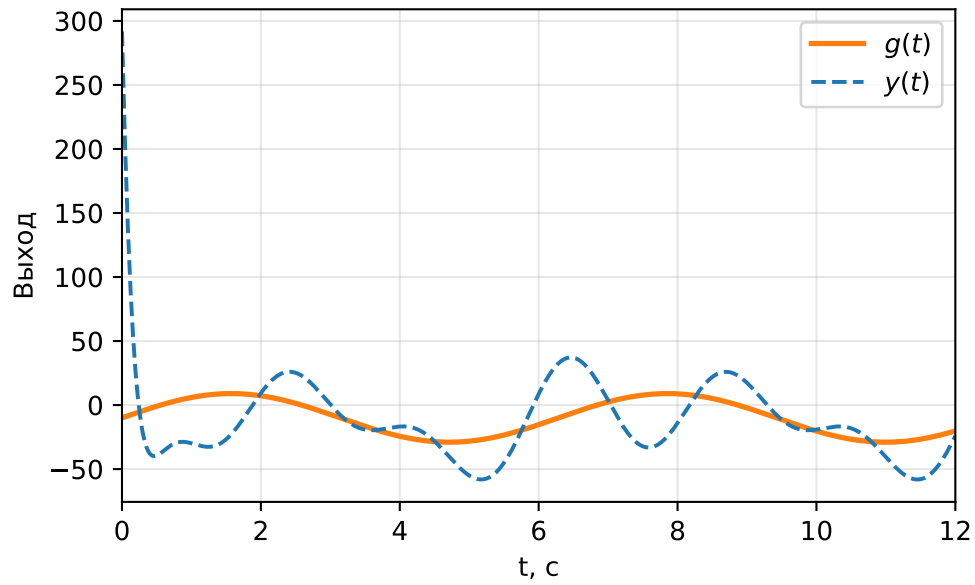


Рисунок 15. Сравнение выхода и задающего воздействия без компенсирующей компоненты

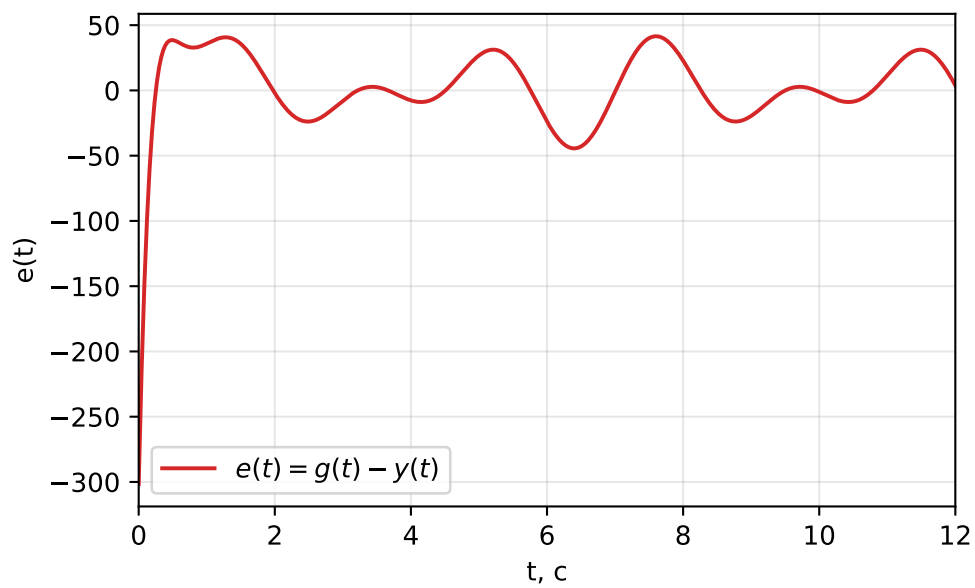


Рисунок 16. Ошибка регулирования без компенсирующей компоненты

В этом режиме регулятор воспроизводит требуемое задающее воздействие, но не компенсирует гармоническое возмущение. Поэтому точность заметно лучше, чем при $u = Kx + K_f w_f$, однако остаточная ошибка из-за внешнего возмущения сохраняется.

д) Полный регулятор $u = Kx + K_g w_g + K_f w_f$

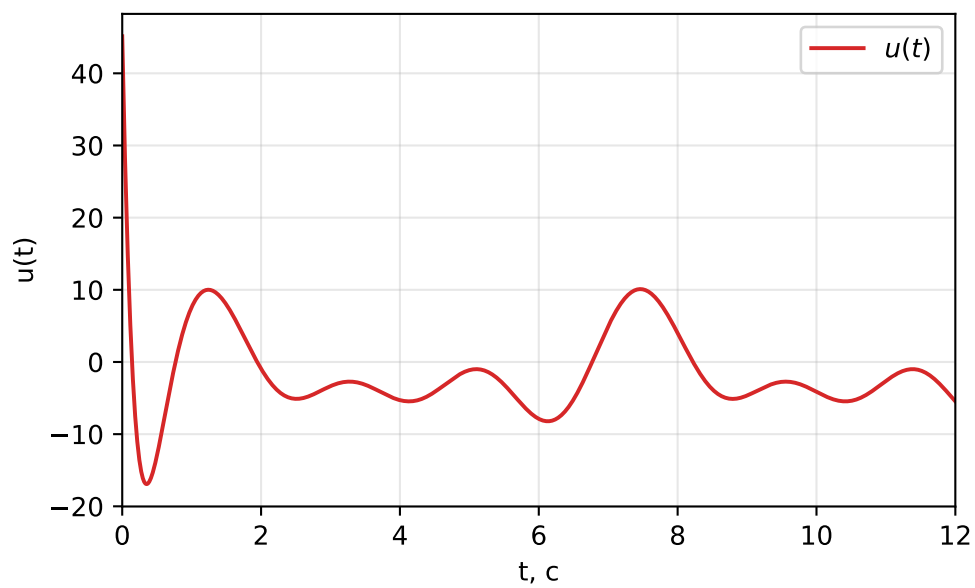


Рисунок 17. Управление полным регулятором

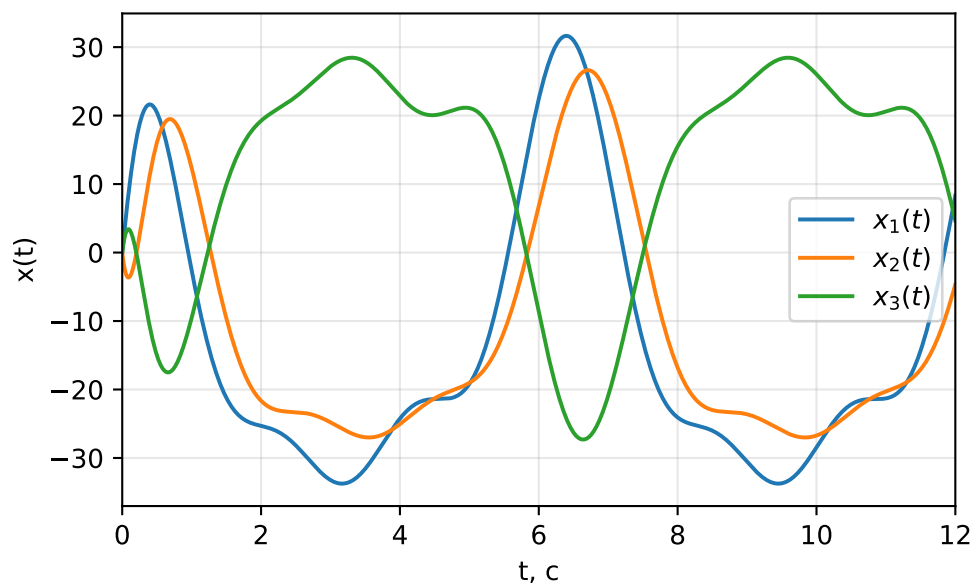


Рисунок 18. Состояние системы при полном регуляторе

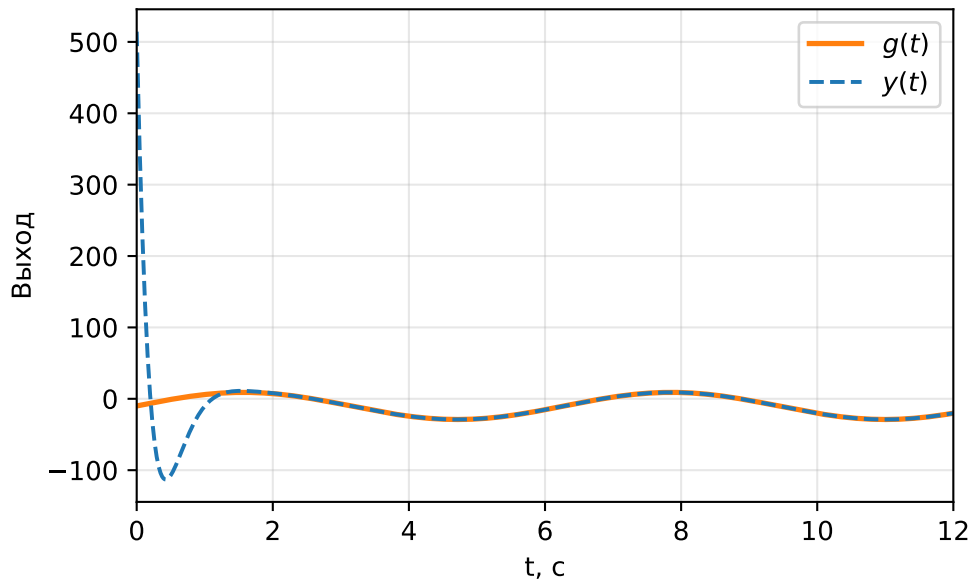


Рисунок 19. Сравнение выхода и задающего воздействия при полном регуляторе

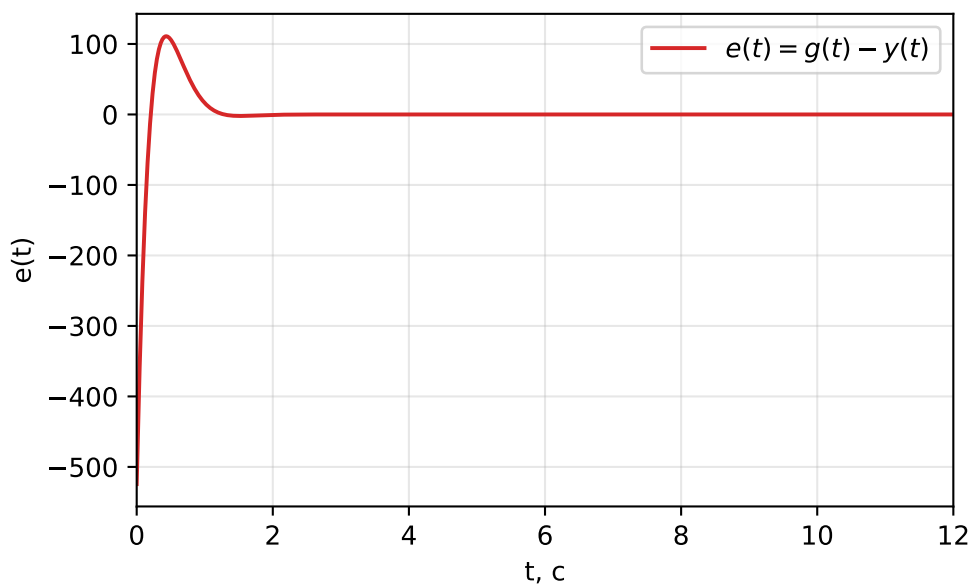


Рисунок 20. Ошибка регулирования при полном регуляторе

Полный регулятор содержит обе внутренние модели: для задающего сигнала и для внешнего возмущения. В результате ошибка быстро убывает и к концу интервала моделирования становится практически нулевой:

$$e(12) \approx 0.$$

Это согласуется с теоретическим условием

$$\lim_{t \rightarrow \infty} |g(t) - y(t)| = 0.$$

Вывод по заданию 1

В первом задании был синтезирован регулятор полной информации на основе матричных уравнений Франкиса-Дэвисона. Для варианта 15 показано, что объект не полностью управляем, но стабилизируем; внешнее возмущение состоит из гармонических мод с частотами 2 и 3; задающее воздействие $g(t) = 19 \sin t - 10$ воспроизводится генератором с собственными числами 0, i и $-i$. После синтеза стабилизирующей матрицы K и решения двух систем уравнений Франкиса-Дэвисона найдены следящая и компенсирующая компоненты K_g и K_f . Моделирование подтвердило, что только полный регулятор $u = Kx + K_g w_g + K_f w_f$ обеспечивает асимптотическое слежение при наличии внешнего возмущения, тогда как частичные варианты закона управления устраняют лишь часть требуемых режимов работы.

Задание 2. Слежение и компенсация по выходу

В этом задании требуется:

- перейти от регулятора полной информации к регулятору по выходу;
- построить наблюдатель задающего воздействия;
- сформировать расширенную систему и синтезировать наблюдатель повышенной размерности;
- сравнить, как использование оценок влияет на качество слежения.

Во втором задании используются найденные в первом задании матрицы

$$K = [-2.963973 \quad -0.547344 \quad -2.331086],$$

$$K_g = [-5.987345 \quad 41.269841 \quad -18.408935], \quad K_f = [30.590395 \quad -21.056621 \quad 22.236775 \quad -9.499760].$$

Требуется построить регулятор по выходу вида

$$u = K\hat{x} + K_g\hat{w}_g + K_f\hat{w}_f,$$

где $\hat{w}_g$ формируется наблюдателем задающего воздействия, а векторы $\hat{x}$ и $\hat{w}_f$ формируются наблюдателем расширенной размерности.

1) Наблюдатель задающего воздействия

Для генератора задания

$$\dot{w}_g = \Gamma_g w_g, \quad g = Y_g w_g$$

выбирается устойчивая желаемая динамика наблюдателя

$$\dot{\tilde{w}}_g = \Gamma_q \tilde{w}_g + Y_q g, \quad \hat{w}_g = Q^{-1} \tilde{w}_g.$$

Выберем

$$\Gamma_q = \begin{bmatrix} -2 & 0 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & -4 \end{bmatrix}, \quad Y_q = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

Тогда матрица смены базиса Q находится из уравнения типа Сильвестра

$$Q\Gamma_g - \Gamma_q Q = Y_q Y_g.$$

Пусть

$$Q = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix}.$$

Подстановка в уравнение дает систему девяти скалярных уравнений, решение которой равно

$$Q = \begin{bmatrix} 7.6 & -3.8 & -5.0 \\ 5.7 & -1.9 & -3.333333 \\ 4.470588 & -1.117647 & -2.5 \end{bmatrix}.$$

Проверка:

$$\max |Q\Gamma_g - \Gamma_q Q - Y_q Y_g| = 3.55 \cdot 10^{-15}.$$

Кроме того,

$$\det Q = -0.353922 \neq 0,$$

поэтому матрица Q обратима и оценка $\hat{w}_g = Q^{-1} \tilde{w}_g$ определена корректно. Динамика ошибки наблюдателя задающего воздействия задается матрицей Γ_q , поэтому

$$\sigma(\Gamma_q) = \{-2, -3, -4\},$$

и ошибка $w_g(t) - \hat{w}_g(t)$ должна экспоненциально затухать.

2) Формирование расширенной системы

Для построения наблюдателя расширенной размерности введем расширенный вектор состояния

$$\xi = \begin{bmatrix} w_f \\ x \end{bmatrix}.$$

Тогда совместная динамика возмущения и объекта записывается как

$$\dot{\xi} = A_{\text{ext}} \xi + B_{\text{ext}} u, \quad y = C_{\text{ext}} \xi + Du,$$

где

$$A_{\text{ext}} = \begin{bmatrix} \Gamma_f & 0_{4 \times 3} \\ B_f Y_f & A \end{bmatrix}, \quad B_{\text{ext}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 2 \\ -2 \\ 2 \end{bmatrix},$$

$$C_{\text{ext}} = [D_f Y_f \quad C].$$

Сначала явно вычислим матрицы $B_f Y_f$ и $D_f Y_f$:

$$B_f Y_f = \begin{bmatrix} 32 & -12 & 8 & -15 \\ 0 & 0 & 0 & 0 \\ -24 & 4 & 4 & 12 \end{bmatrix},$$

$$D_f Y_f = [56 \quad -56 \quad 84 \quad -21].$$

Следовательно,

$$A_{\text{ext}} = \begin{bmatrix} -40 & 16 & 9 & 7 & 0 & 0 & 0 \\ -64 & 25 & 14 & 12 & 0 & 0 & 0 \\ -26 & 11 & 7 & 3 & 0 & 0 & 0 \\ -48 & 18 & 14 & 8 & 0 & 0 & 0 \\ 32 & -12 & 8 & -15 & 4 & 1 & 6 \\ 0 & 0 & 0 & 0 & 2 & 0 & 2 \\ -24 & 4 & 4 & 12 & -2 & -2 & -4 \end{bmatrix},$$

$$B_{\text{ext}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 2 \\ -2 \\ 2 \end{bmatrix},$$

$$C_{\text{ext}} = [56 \quad -56 \quad 84 \quad -21 \quad 2 \quad -2 \quad 1].$$

Проверим наблюдаемость расширенной системы. Ранг матрицы наблюдаемости равен

$$\text{rank}(O_{\text{ext}}) = 7,$$

то есть расширенная система полностью наблюдаема.

3) Синтез наблюдателя расширенной размерности

Для расширенной системы используем стандартный наблюдатель полной размерности в форме из лекции:

$$\dot{\hat{\xi}} = A_{\text{ext}} \hat{\xi} + B_{\text{ext}} u + L(\hat{y} - y) + L D u,$$

$$\hat{y} = C_{\text{ext}} \hat{\xi}.$$

Эквивалентно,

$$\dot{\hat{\xi}} = (A_{\text{ext}} + L C_{\text{ext}}) \hat{\xi} + (B_{\text{ext}} + L D) u - L y.$$

Матрицу L найдем методом задания спектра, задав желаемые собственные числа ошибки наблюдателя

$$\sigma(A_{\text{ext}} + L C_{\text{ext}}) = \{-2, -3, -4, -5, -6, -7, -8\}.$$

В результате получаем

$$L = \begin{bmatrix} 119.208300 \\ 184.798050 \\ 85.536496 \\ 140.898435 \\ 2001.362925 \\ 1530.501033 \\ -1529.896281 \end{bmatrix}.$$

Проверка показывает, что собственные числа матрицы ошибки действительно равны

$$\sigma(A_{\text{ext}} + LC_{\text{ext}}) = \{-8, -7.000001, -5.999998, -5.000002, -4, -3, -2\}.$$

Следовательно, ошибки $\xi(t) - \hat{\xi}(t)$ экспоненциально затухают.

4) Моделирование замкнутой системы

Проводим моделирование на интервале $t \in [0, 6]$ при нулевых начальных условиях наблюдателей:

$$\tilde{w}_g(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{\xi}(0) = 0_{7 \times 1}.$$

Вводятся ошибки

$$e_f(t) = \begin{bmatrix} w_f(t) \\ x(t) \end{bmatrix} - \begin{bmatrix} \hat{w}_f(t) \\ \hat{x}(t) \end{bmatrix},$$

$$e_g(t) = w_g(t) - \hat{w}_g(t), \quad e(t) = g(t) - y(t).$$

Сводные численные характеристики моделирования:

Показатель	Значение
$\max u(t) $	2774.622
$\max x(t) $	649.376
$\max e_f(t) $	2445.526
$\max e_g(t) $	2.305
$e(6)$	-0.31647

Таблица 2. Характеристики моделирования для задания 2

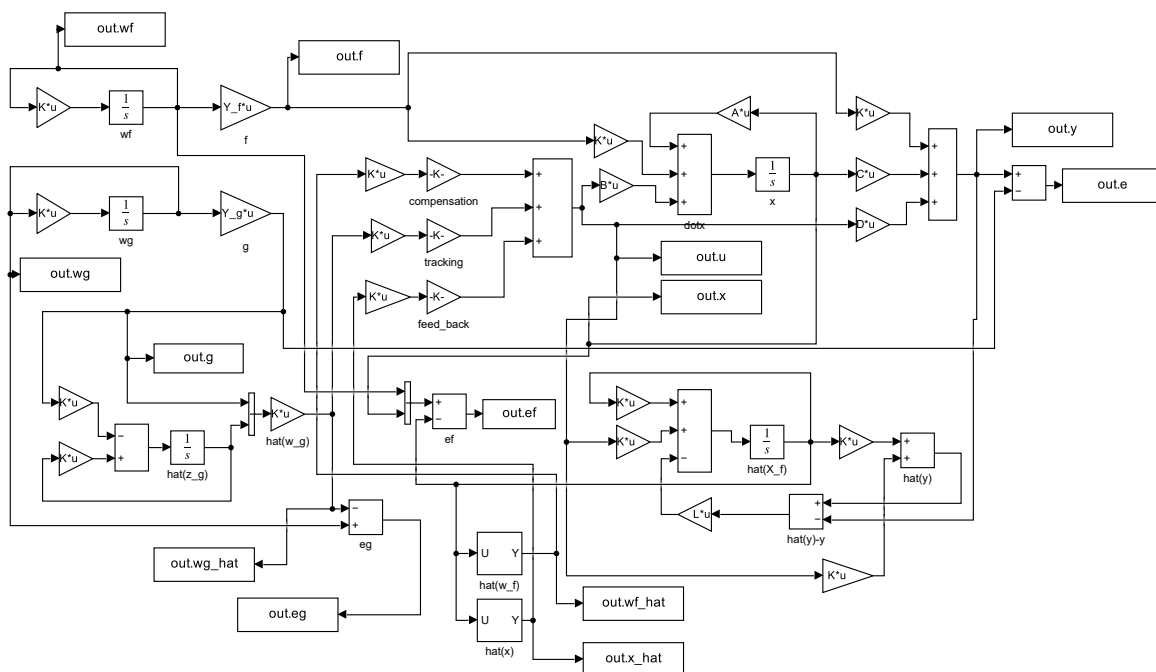


Рисунок 21. Структурная схема моделирования регулятора по выходу

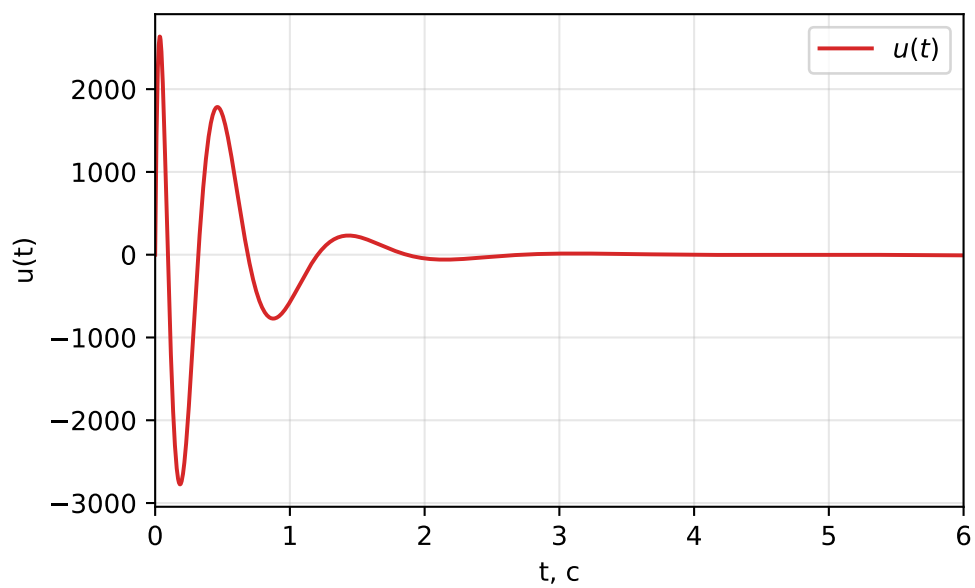


Рисунок 22. Управление регулятора по выходу

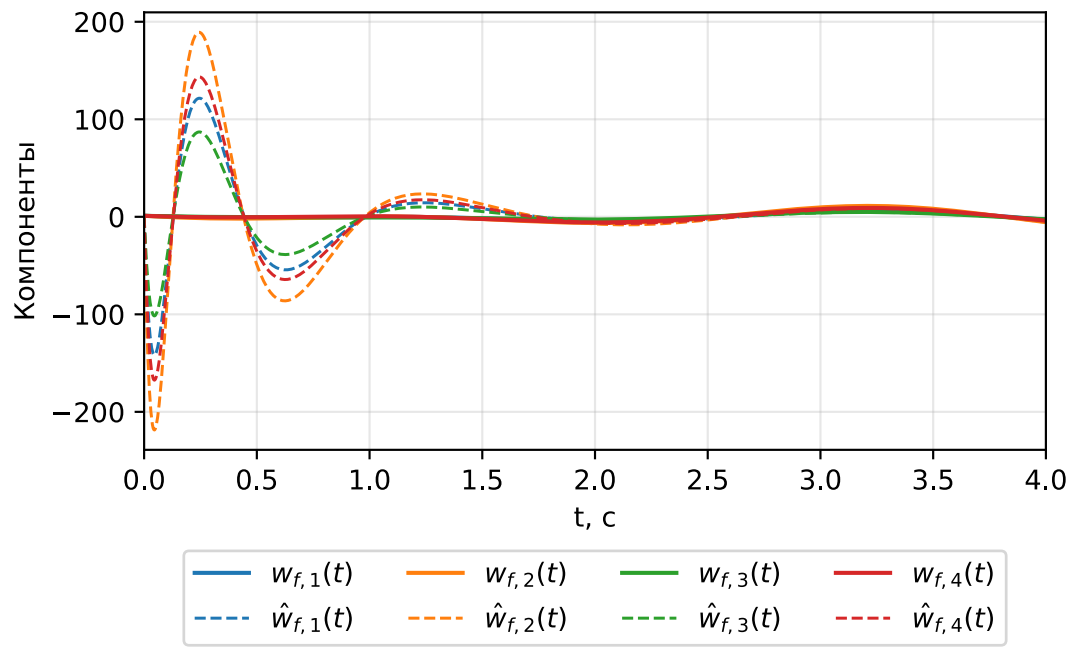


Рисунок 23. Сравнение компонент $w_f(t)$ и $\hat{w}_f(t)$

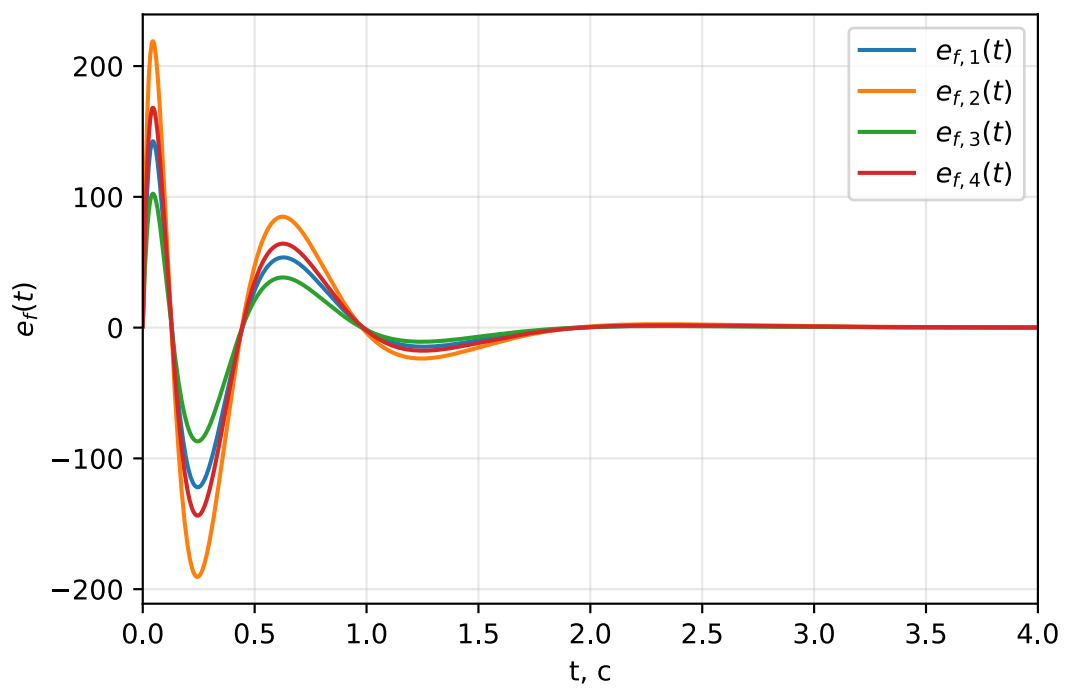


Рисунок 24. Ошибка оценки возмущения $w_f(t) - \hat{w}_f(t)$

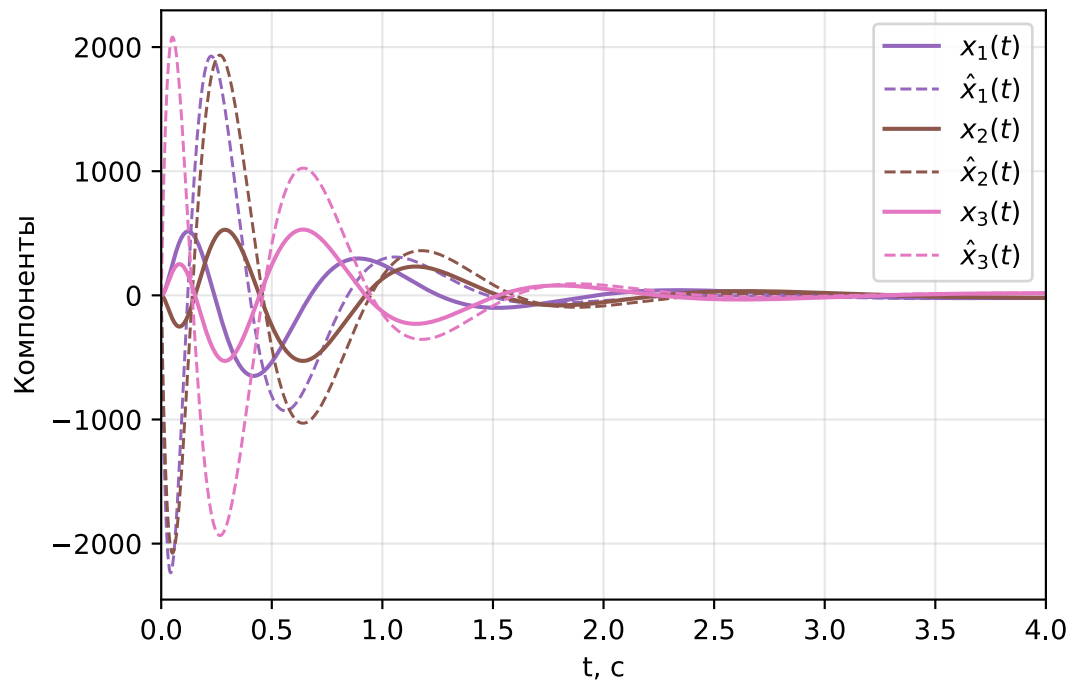


Рисунок 25. Сравнение компонент $x(t)$ и $\hat{x}(t)$

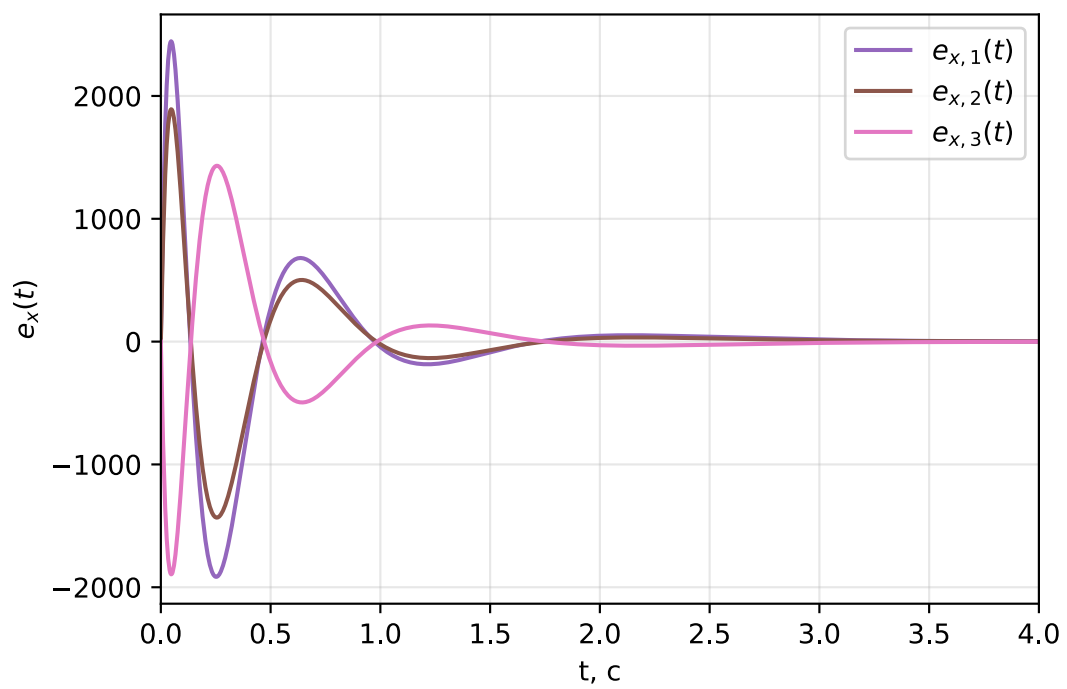


Рисунок 26. Ошибка оценки состояния $x(t) - \hat{x}(t)$

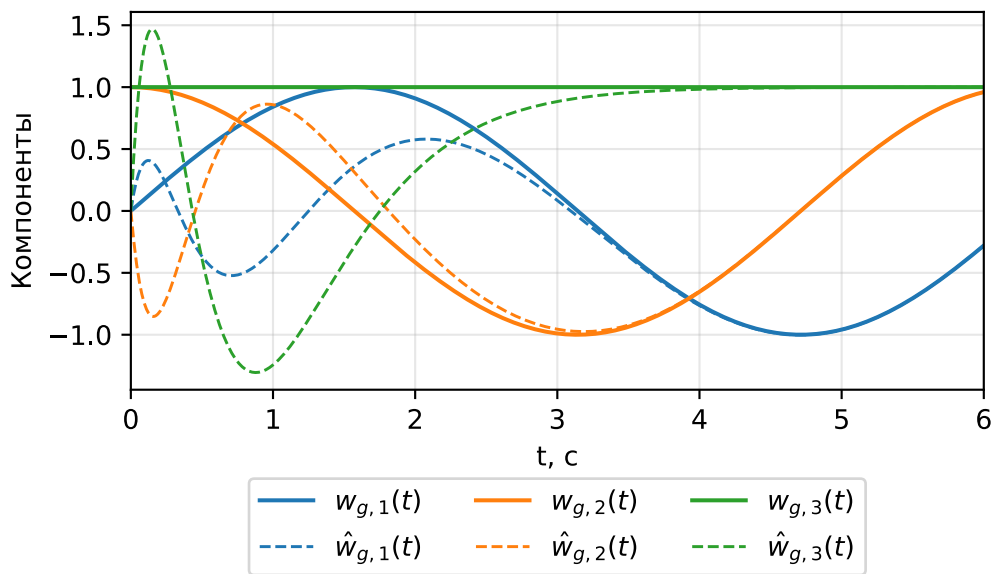


Рисунок 27. Сравнение компонент $w_g(t)$ и $\hat{w}_g(t)$

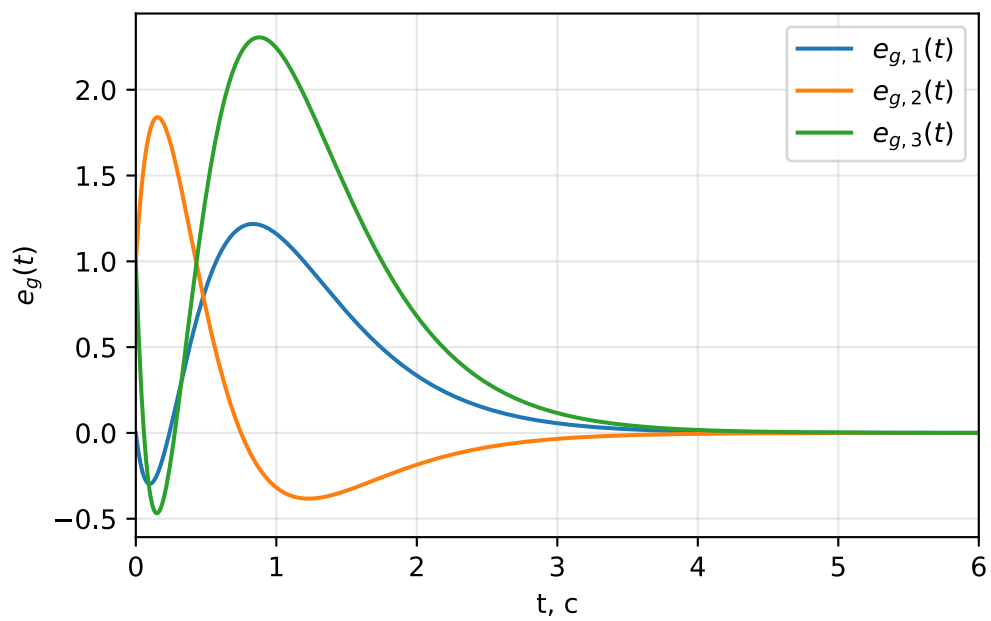


Рисунок 28. Ошибка наблюдателя задающего воздействия $e_g(t)$

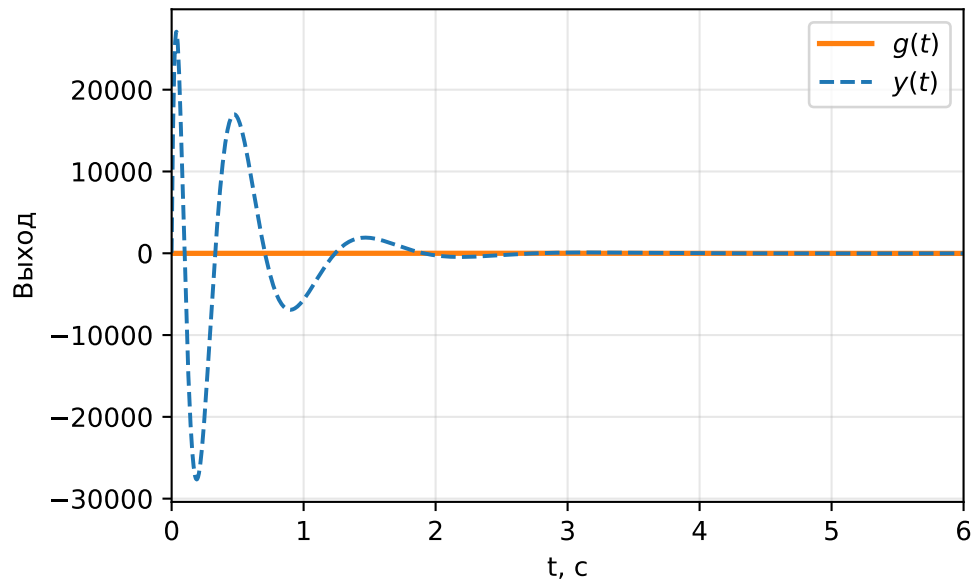


Рисунок 29. Сопоставление выхода $y(t)$ и задания $g(t)$

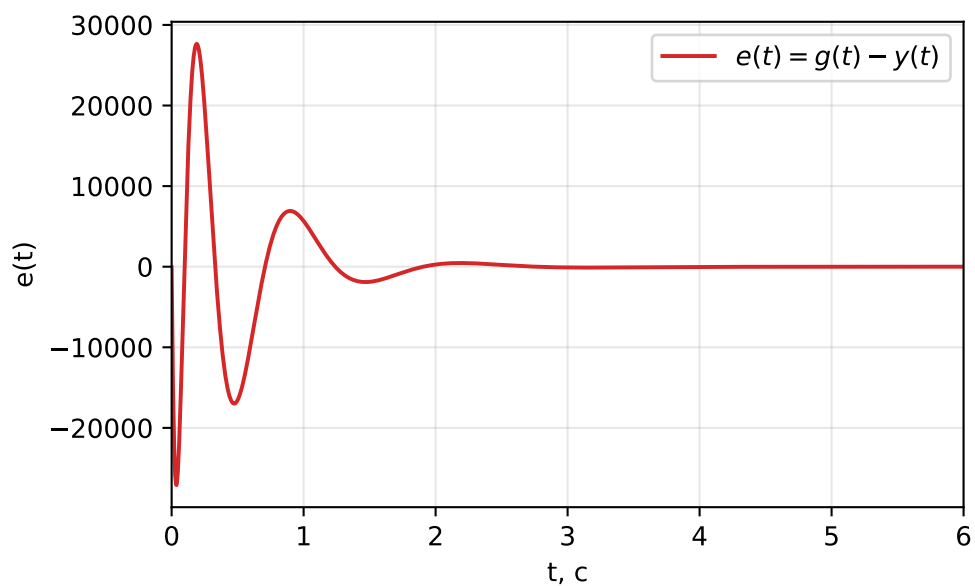


Рисунок 30. Ошибка управления $e(t) = g(t) - y(t)$

Графики показывают следующее.

1. Наблюдатель задающего воздействия корректно восстанавливает вектор $w_g(t)$: все компоненты ошибки $e_g(t)$ затухают, а к концу моделирования $\hat{w}_g(t)$ практически совпадает с $w_g(t)$.
2. Наблюдатель расширенной размерности восстанавливает как возмущение $w_f(t)$, так и состояние объекта $x(t)$. Ошибка $e_f(t)$ на начальном участке заметно больше, чем в первом задании, поскольку теперь управление строится по оценкам, а не по точным значениям переменных.

3. Несмотря на существенный стартовый выброс в управлении и в ошибках наблюдения, к концу интервала ошибка слежения становится малой:

$$\max_{t \in [5.4, 6]} |e(t)| = 1.1839, \quad e(6) \approx -3.16 \cdot 10^{-1}.$$

С практической точки зрения это означает, что переход от полной информации к управлению по выходу не разрушает саму возможность слежения, но делает реализацию значительно более чувствительной к качеству оценивания. Поэтому во втором задании определяющим фактором становится уже не только сам регулятор, но и точность восстановления недоступных переменных.

Поэтому закон управления по выходу

$$u = K\hat{x} + K_g\hat{w}_g + K_f\hat{w}_f$$

обеспечивает тот же качественный результат, что и регулятор полной информации из первого задания, но ценой более выраженного переходного процесса.

Вывод по заданию 2

Во втором задании регулятор полной информации из первого задания был дополнен двумя наблюдателями: наблюдателем задающего воздействия и наблюдателем расширенной размерности. По лекционной схеме через уравнение типа Сильвестра была найдена матрица Q наблюдателя сигнала задания, а для расширенной системы $\begin{bmatrix} w_f \\ x \end{bmatrix}$ построен наблюдатель полной размерности с матрицей L . Моделирование показало, что обе оценки сходятся к истинным значениям, а ошибка управления $e(t) = g(t) - y(t)$ затухает к нулю. По сравнению с первым заданием переходный процесс стал существенно жестче, поскольку управление формируется по оценкам, однако требуемое свойство асимптотического слежения сохраняется.

Задание 3. Наблюдатели возмущения

В этом задании требуется:

- построить два специальных наблюдателя возмущения;
- замкнуть систему регулятором с оценками $\hat{w}_f$ и $\hat{w}_g$;
- сравнить оба варианта между собой и с наблюдателем расширенной размерности из задания 2;
- оценить, какой способ наблюдения возмущения для данной системы оказывается наиболее удачным.

В третьем задании используются матрицы K , K_g , K_f из первого задания и матрица Q наблюдателя задающего воздействия из второго задания. В отличие от задания 2, вектор состояния объекта $x(t)$ считается доступным к измерению, поэтому регулятор имеет вид

$$u = Kx + K_g\hat{w}_g + K_f\hat{w}_f.$$

Необходимо рассмотреть два варианта наблюдателя возмущения:

1. наблюдатель по состоянию;
2. наблюдатель по выходу.

1) Наблюдатель возмущения по состоянию

По лекции этот наблюдатель строится в форме

$$\hat{w}_f = z + L\tilde{C}x,$$

$$\dot{z} = Fz + FL\tilde{C}x - L\tilde{C}Ax - L\tilde{C}Bu,$$

где матрица $\tilde{C}$ должна удовлетворять условию

$$\tilde{C}B_f = I_2.$$

Сначала проверим, что такой левый обратный множитель существует. Матрица B_f имеет размер 3×2 и ранг 2, поэтому можно взять

$$\tilde{C} = (B_f^T B_f)^{-1} B_f^T.$$

Вычислим:

$$B_f^T B_f = \begin{bmatrix} 8 & -2 \\ -2 & 1 \end{bmatrix}, \quad (B_f^T B_f)^{-1} = \begin{bmatrix} \frac{1}{4} & \frac{1}{2} \\ \frac{1}{2} & 2 \end{bmatrix}.$$

Тогда

$$\tilde{C} = \begin{bmatrix} 0 & 0 & \frac{1}{2} \\ 1 & 0 & 1 \end{bmatrix},$$

и действительно

$$\tilde{C}B_f = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2.$$

Далее по лекции необходимо обеспечить желаемую динамику ошибки наблюдения

$$\dot{\tilde{w}}_f = F\tilde{w}_f, \quad F = \Gamma_f - LY_f.$$

Выберем желаемый спектр

$$\sigma(F) = \{-2, -3, -4, -5\}.$$

Матрица L находится как матрица дуального наблюдателя для пары (Γ_f, Y_f) , то есть из условия

$$\sigma(\Gamma_f - LY_f) = \{-2, -3, -4, -5\}.$$

Получаем

$$L = \begin{bmatrix} -5.665509 & 2.664615 \\ -7.909763 & 4.249765 \\ -3.223940 & 2.370847 \\ -6.263253 & 3.302717 \end{bmatrix}.$$

Тогда

$$F = \Gamma_f - LY_f, \quad \sigma(F) = \{-5, -2, -3, -4\}.$$

Следовательно, ошибка наблюдателя по состоянию должна экспоненциально затухать.

2) Наблюдатель возмущения по выходу

Так как в нашей системе

$$y = Cx + Du + D_f f,$$

при измеримом x и известном u можно сформировать сигнал

$$\hat{f} = y - Cx - Du = D_f f = D_f Y_f w_f.$$

По лекции наблюдатель по выходу строится в форме

$$\dot{\tilde{w}}_f = \Gamma \tilde{w}_f + Y \hat{f}, \quad \hat{w}_f = Q^{-1} \tilde{w}_f.$$

Выберем желаемую матрицу наблюдателя

$$\Gamma = \begin{bmatrix} -2 & 0 & 0 & 0 \\ 0 & -3 & 0 & 0 \\ 0 & 0 & -4 & 0 \\ 0 & 0 & 0 & -5 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Матрица Q находится из уравнения типа Сильвестра

$$Q\Gamma_f - \Gamma Q = Y D_f Y_f.$$

Сначала вычислим

$$D_f Y_f = [56 \quad -56 \quad 84 \quad -21].$$

Тогда решение уравнения имеет вид

$$Q = \begin{bmatrix} 2.423077 & -18.576923 & 13.192308 & 14.538462 \\ 3.589744 & -15.256410 & 14.717949 & 8.435897 \\ 4.2 & -12.6 & 14.0 & 4.9 \\ 4.430020 & -10.606491 & 12.722110 & 2.853955 \end{bmatrix}.$$

Проверка:

$$\max |Q\Gamma_f - \Gamma Q - Y D_f Y_f| = 7.67 \cdot 10^{-13}.$$

Кроме того,

$$\det Q = -0.187315 \neq 0,$$

поэтому матрица Q обратима и оценка $\hat{w}_f = Q^{-1} \tilde{w}_f$ определена корректно.

3) Моделирование и сравнение

В обоих случаях используется тот же наблюдатель задающего воздействия, что и во втором задании, а начальные условия наблюдателей принимаются нулевыми.

Для сравнения сведем характерные показатели в общую таблицу. Для задания 2 оставлены итоговые значения наблюдателя расширенной размерности.

Вариант	$\max u $	$\max x $	$\max e_f $	Ошибка в конце интервала
Задание 2	2774.622	649.376	2445.526	-0.31647
По состоянию	28.241	31.653	7.443	-0.00133
По выходу	14.442	31.654	16.324	-0.00169

Таблица 3. Сравнение наблюдателей возмущения

Из этой таблицы видно главное. Наблюдатель расширенной размерности из задания 2 формально решает ту же задачу, но делает это ценой очень тяжелого переходного процесса. Специальные наблюдатели возмущения используют дополнительную структурную информацию о системе, поэтому дают заметно более мягкое управление и меньшие ошибки. При этом наблюдатель по состоянию оказывается самым точным, а наблюдатель по выходу дает более умеренное управление, но несколько уступает по качеству оценивания.

а) Наблюдатель возмущения по состоянию

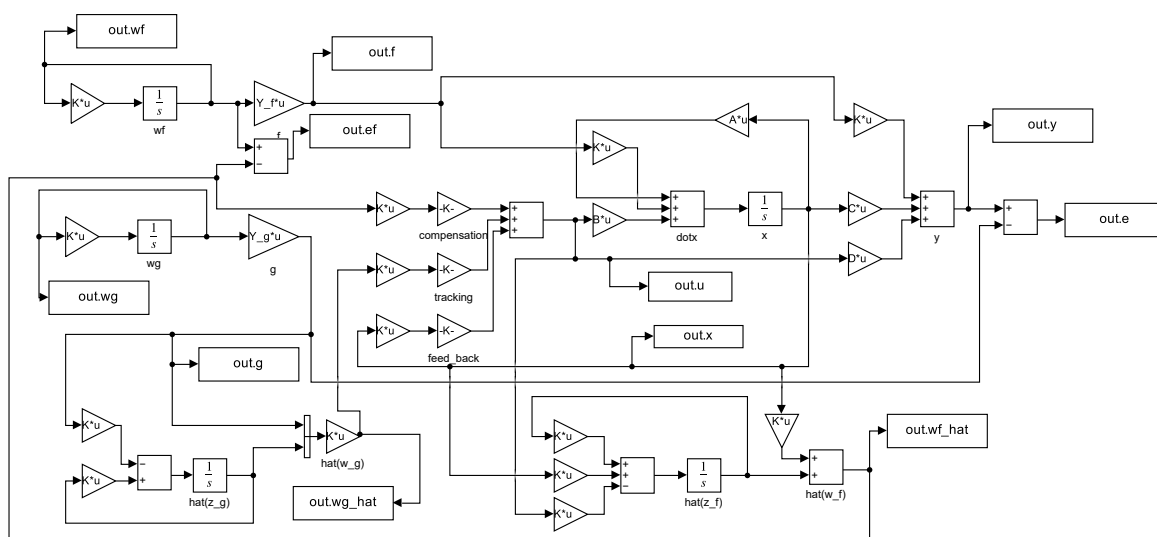


Рисунок 31. Структурная схема моделирования наблюдателя возмущения по состоянию

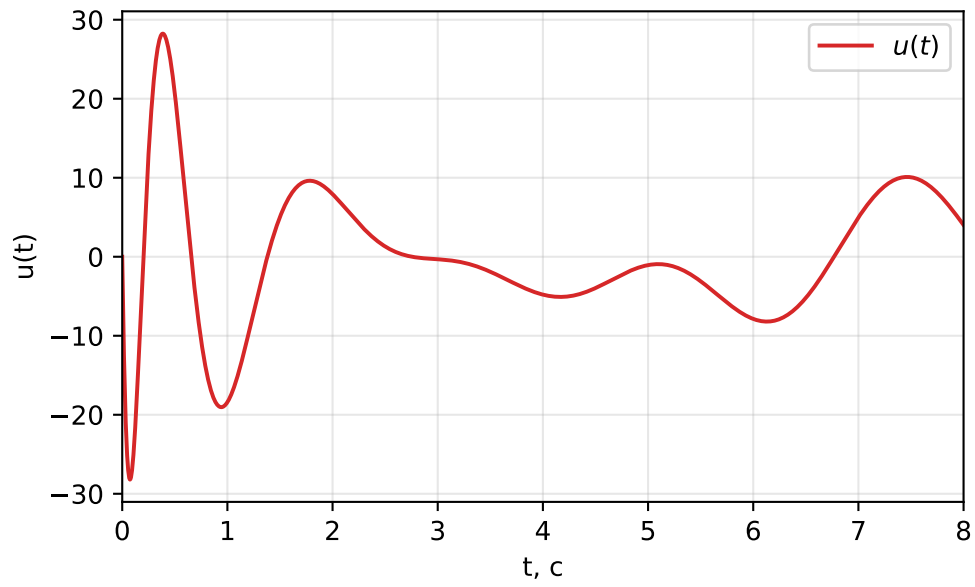


Рисунок 32. Управление при наблюдателе возмущения по состоянию

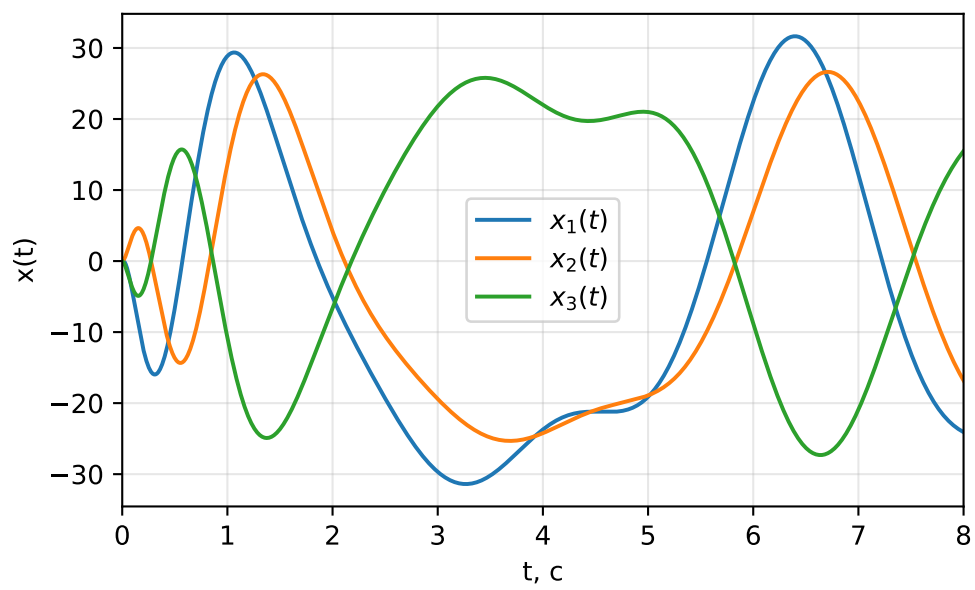


Рисунок 33. Состояние системы при наблюдателе возмущения по состоянию

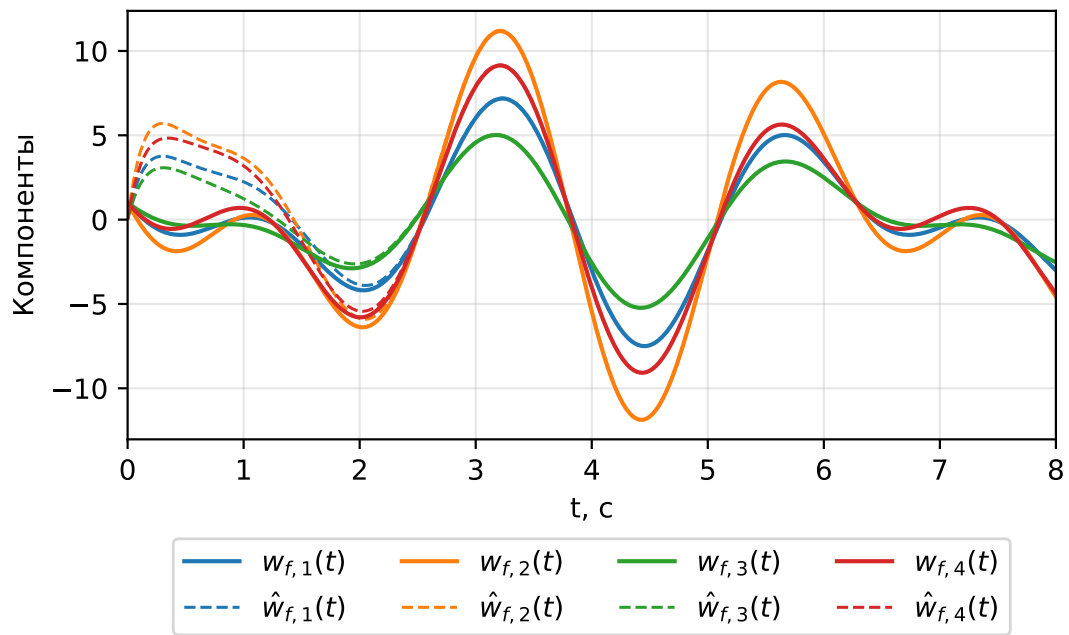


Рисунок 34. Сравнение компонент $w_f(t)$ и $\hat{w}_f(t)$ для наблюдателя по состоянию

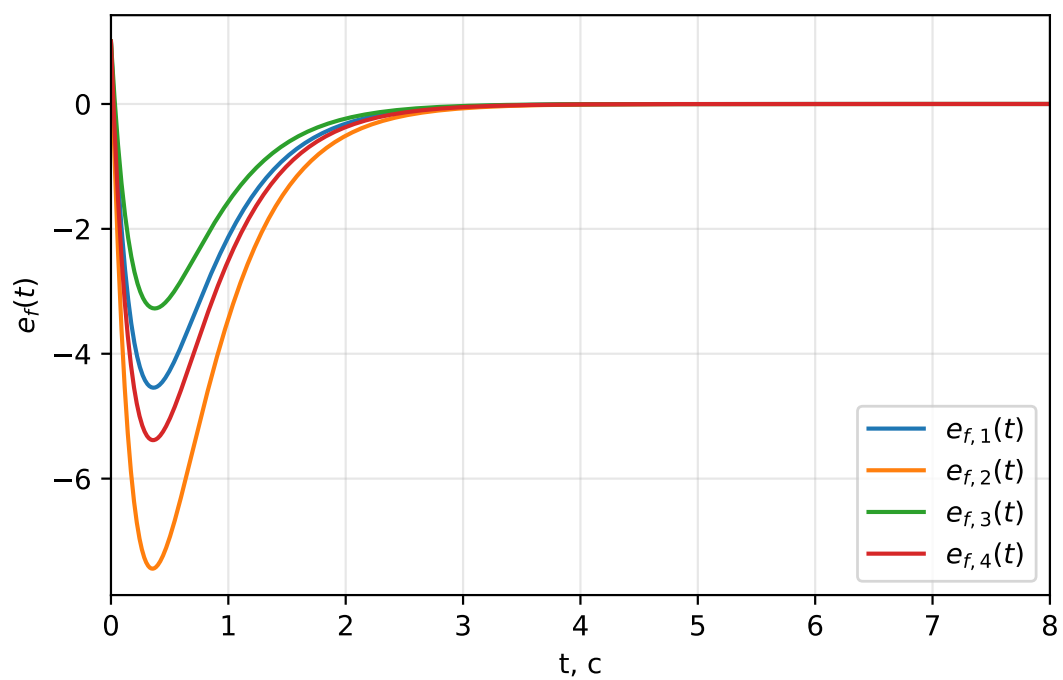


Рисунок 35. Ошибка наблюдателя по состоянию $e_f(t)$

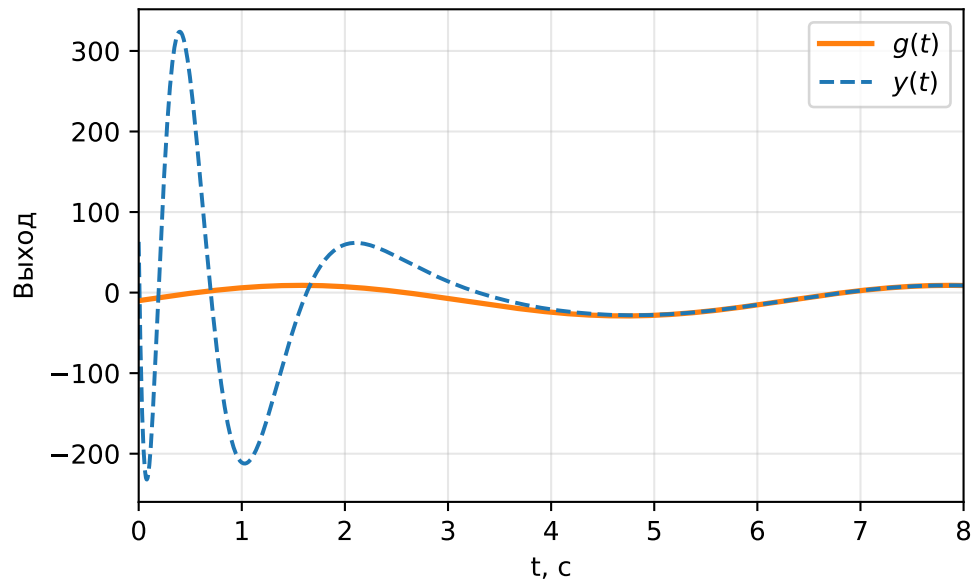


Рисунок 36. Сопоставление выхода и задания для наблюдателя по состоянию

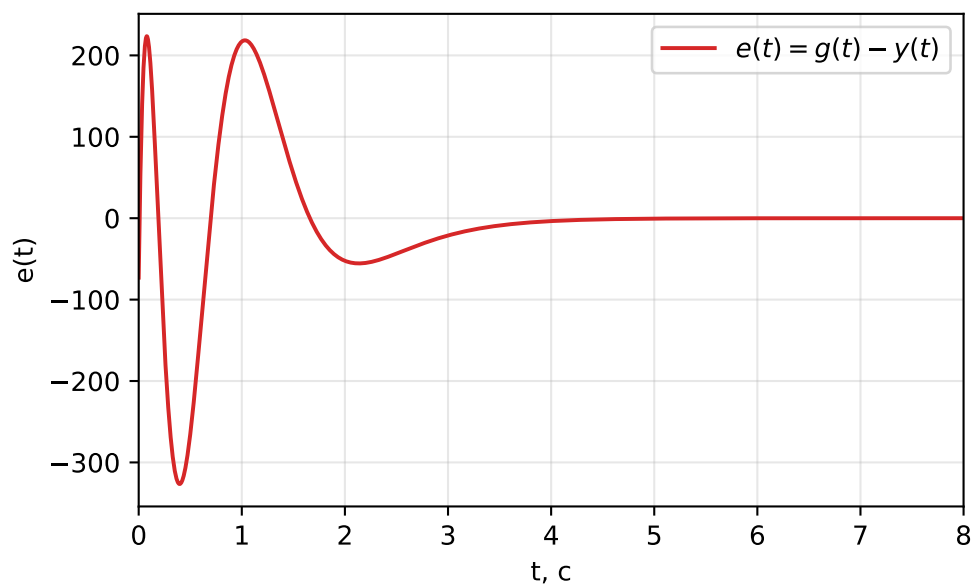


Рисунок 37. Ошибка управления для наблюдателя по состоянию

Для наблюдателя по состоянию получаем

$$\max|u(t)| = 28.241, \quad \max|e_f(t)| = 7.443, \quad e(8) \approx -1.33 \cdot 10^{-3}.$$

Этот вариант дает наилучшее качество оценки возмущения и наименьшую ошибку управления из всех трех рассмотренных реализаций.

б) Наблюдатель возмущения по выходу

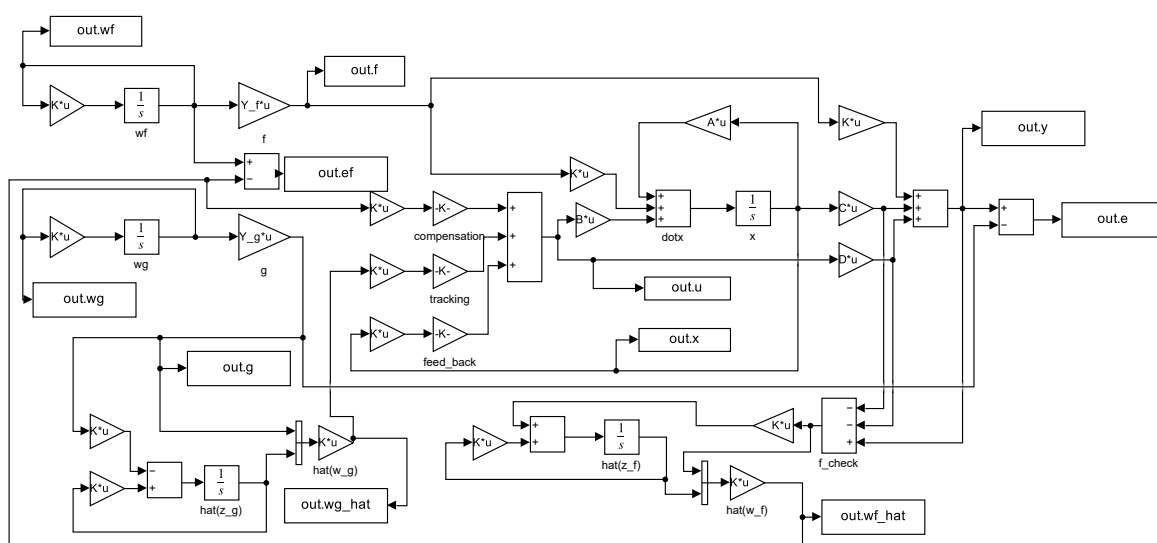


Рисунок 38. Структурная схема моделирования наблюдателя возмущения по выходу

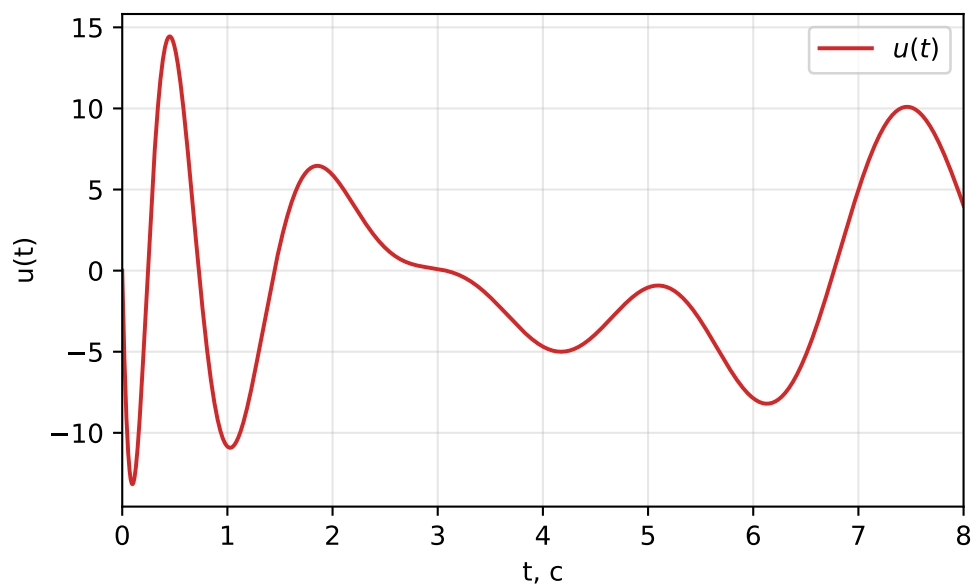


Рисунок 39. Управление при наблюдателе возмущения по выходу

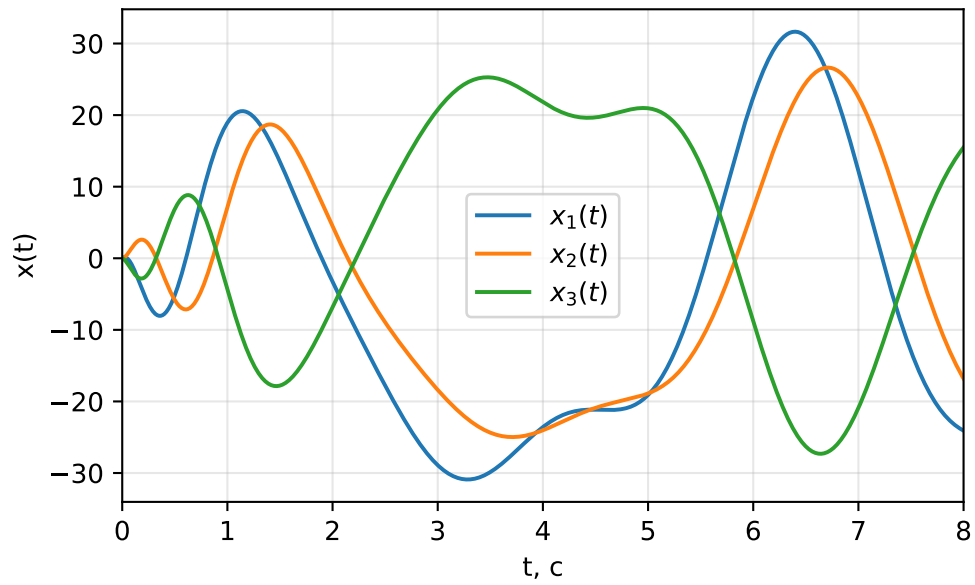


Рисунок 40. Состояние системы при наблюдателе возмущения по выходу

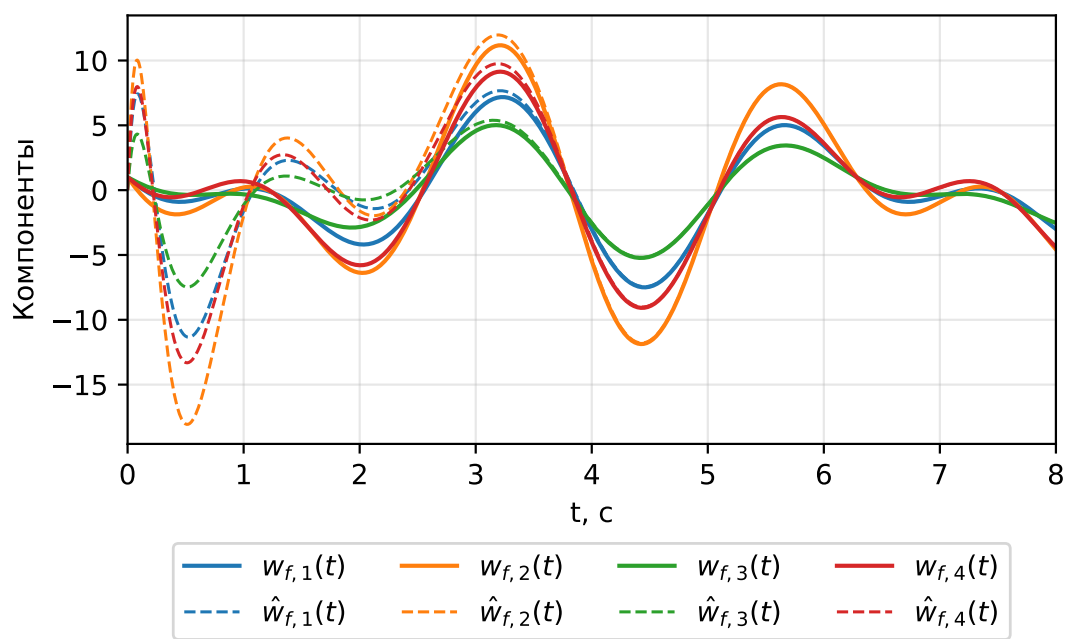


Рисунок 41. Сравнение компонент $w_f(t)$ и $\hat{w}_f(t)$ для наблюдателя по выходу

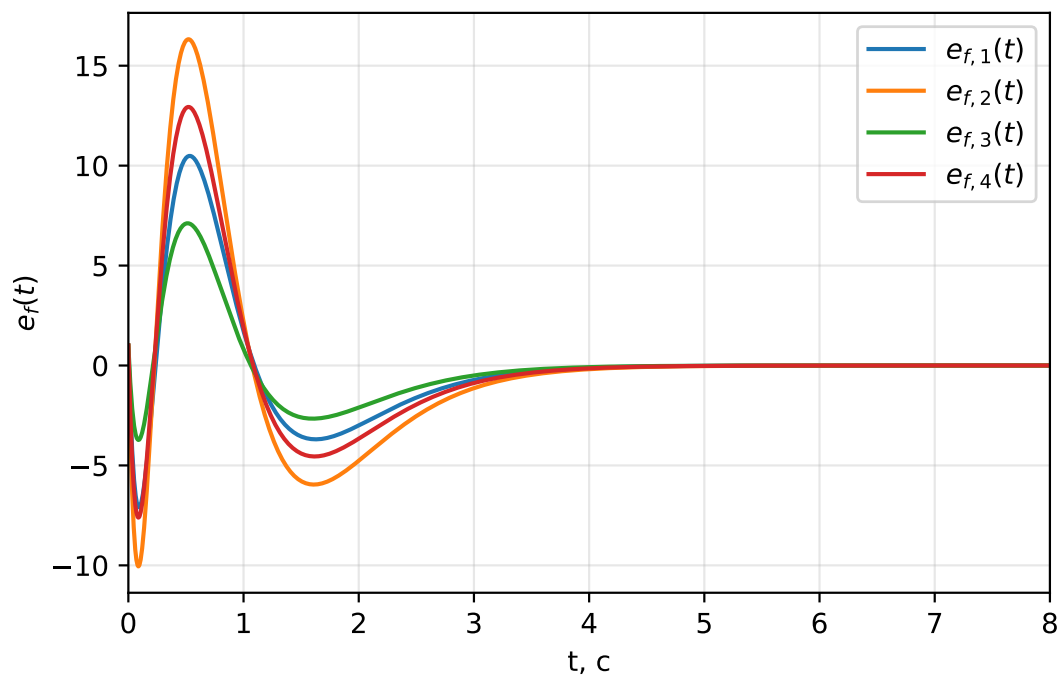


Рисунок 42. Ошибка наблюдателя по выходу $e_f(t)$

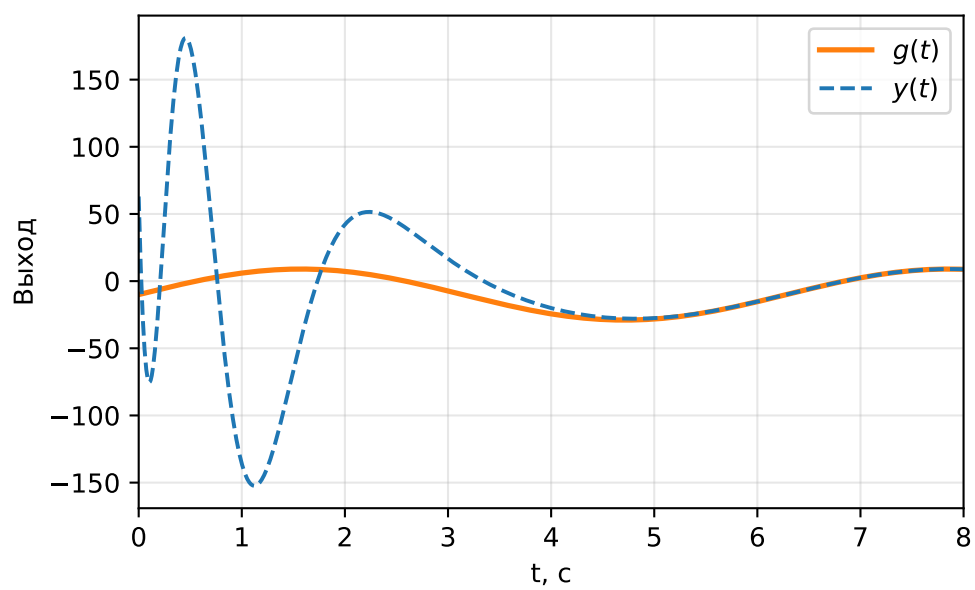


Рисунок 43. Сопоставление выхода и задания для наблюдателя по выходу

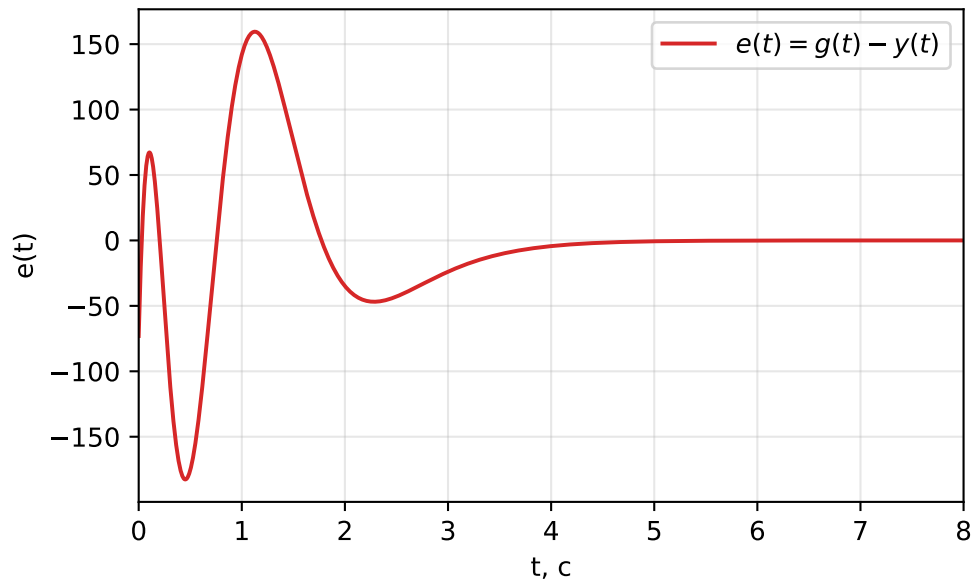


Рисунок 44. Ошибка управления для наблюдателя по выходу

Для наблюдателя по выходу получаем

$$\max|u(t)| = 14.442, \quad \max|e_f(t)| = 16.324, \quad e(8) \approx -1.69 \cdot 10^{-3}.$$

Этот вариант требует меньшего управления, но точность оценки возмущения ниже, чем у наблюдателя по состоянию. Тем не менее ошибка слежения также затухает к нулю.

Вывод по заданию 3

В третьем задании были исследованы два специальных наблюдателя возмущения при измеримом состоянии объекта. Наблюдатель по состоянию использует левый обратный множитель к B_f и дает наиболее точную оценку возмущения, а наблюдатель по выходу строится по скалярному сигналу $\hat{f} = y - Cx - Du$ и обеспечивает более мягкое управление. Оба подхода оказались существенно эффективнее наблюдателя расширенной размерности из второго задания: и амплитуда управления, и ошибки наблюдения, и ошибка слежения заметно уменьшились. Наиболее точным вариантом для данной системы оказался наблюдатель возмущения по состоянию.

Общий вывод

Лабораторная работа показала, что задача слежения и компенсации в линейных системах по существу сводится к двум взаимосвязанным вопросам: нужно встроить в регулятор внутренние модели внешних воздействий и затем выбрать такую структуру наблюдения, которая позволит реализовать этот регулятор по реально доступным измерениям.

На данной системе все рассмотренные подходы подтвердили одно и то же принципиальное свойство: при корректном включении моделей задающего сигнала и возмущения ошибка $e(t) = g(t) - y(t)$ действительно может быть сведена к нулю. При этом основное различие между вариантами заключается уже не в самом факте достижения слежения, а в качестве переходного процесса. Полная информация дает наилучшую теоретическую картину, управление по выходу через наблюдатель расширенной размерности сохраняет требуемый результат, но оказывается наиболее тяжелым по пикам сигналов, а специальные

наблюдатели возмущения позволяют заметно упростить реализацию и улучшить практические показатели.

Итоговый вывод состоит в том, что для таких задач решающим является не только синтез регулятора, но и способ восстановления недоступных внешних воздействий. Именно структура наблюдателя определяет, насколько дорого с точки зрения управления и переходного процесса будет стоить реализация принципа внутренней модели.

Приложение

Код для задания 1

```
1  import json
2  import os
3  from pathlib import Path
4
5  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
6
7  import control as ct
8  import matplotlib.pyplot as plt
9  import numpy as np
10 from scipy.integrate import solve_ivp
11
12 BASE_DIR = Path(__file__).resolve().parents[1]
13 PLOTS_DIR = BASE_DIR / "plots_task1"
14 RESULTS_PATH = Path(__file__).with_name("task1_results.json")
15
16 A = np.array([
17     [4.0, 1.0, 6.0],
18     [2.0, 0.0, 2.0],
19     [-2.0, -2.0, -4.0],
20 ])
21 B = np.array([[2.0], [-2.0], [2.0]])
22 B_F = np.array([
23     [-2.0, 1.0],
24     [0.0, 0.0],
25     [2.0, 0.0],
26 ])
27 C = np.array([[2.0, -2.0, 1.0]])
28 D = np.array([[10.0]])
29 D_F = np.array([[0.0, 7.0]])
30
31 GAMMA_F = np.array([
32     [-40.0, 16.0, 9.0, 7.0],
33     [-64.0, 25.0, 14.0, 12.0],
34     [-26.0, 11.0, 7.0, 3.0],
35     [-48.0, 18.0, 14.0, 8.0],
36 ])
37 Y_F = np.array([
38     [-12.0, 2.0, 2.0, 6.0],
39     [8.0, -8.0, 12.0, -3.0],
40 ])
41
42 GAMMA_G = np.array([
```

```

43     [0.0, 1.0, 0.0],
44     [-1.0, 0.0, 0.0],
45     [0.0, 0.0, 0.0],
46 ]
47 Y_G = np.array([[19.0, 0.0, -10.0]])
48
49 X0 = np.zeros(3)
50 WF0 = np.ones(4)
51 WG0 = np.array([0.0, 1.0, 1.0])
52 T = np.linspace(0.0, 12.0, 3601)
53
54 Q_LQR = np.eye(3)
55 R_LQR = np.array([[1.0]])
56
57 COLORS = {
58     "u": "tab:red",
59     "x1": "tab:blue",
60     "x2": "tab:orange",
61     "x3": "tab:green",
62     "f1": "tab:purple",
63     "f2": "tab:brown",
64     "g": "tab:orange",
65     "y": "tab:blue",
66     "e": "tab:red",
67 }
68
69 SCENARIOS = {
70     "open": {
71         "label": r"$u = 0$",
72         "Kx": np.zeros((1, 3)),
73         "Kg": np.zeros((1, 3)),
74         "Kf": np.zeros((1, 4)),
75     },
76     "feedback": {
77         "label": r"$u = Kx$",
78         "Kx": None,
79         "Kg": np.zeros((1, 3)),
80         "Kf": np.zeros((1, 4)),
81     },
82     "no_tracking": {
83         "label": r"$u = Kx + K_f w_f$",
84         "Kx": None,
85         "Kg": np.zeros((1, 3)),
86         "Kf": None,
87     },
88     "no_compensation": {

```

```

89     "label": r"$u = Kx + K_g w_g$",
90     "Kx": None,
91     "Kg": None,
92     "Kf": np.zeros((1, 4)),
93 },
94 "full": {
95     "label": r"$u = Kx + K_g w_g + K_f w_f$",
96     "Kx": None,
97     "Kg": None,
98     "Kf": None,
99 },
100 }
101
102
103 def round_matrix(mat, digits=6):
104     return np.round(np.asarray(mat, dtype=float), digits).tolist()
105
106
107 def serialize_values(values, digits=6):
108     out = []
109     for value in np.asarray(values).flatten():
110         if abs(np.imag(value)) < 1e-9:
111             out.append(round(float(np.real(value)), digits))
112         else:
113             out.append({
114                 "re": round(float(np.real(value)), digits),
115                 "im": round(float(np.imag(value)), digits),
116             })
117     return out
118
119
120 def serialize_matrix(mat, digits=6):
121     arr = np.asarray(mat)
122     out = []
123     for row in arr:
124         item = []
125         for value in row:
126             if abs(np.imag(value)) < 1e-9:
127                 item.append(round(float(np.real(value)), digits))
128             else:
129                 item.append({
130                     "re": round(float(np.real(value)), digits),
131                     "im": round(float(np.imag(value)), digits),
132                 })
133         out.append(item)
134     return out

```

```

135
136
137 def solve_fd(gamma, rhs_top, rhs_bottom, acl, ccl):
138     n = A.shape[0]
139     q = gamma.shape[0]
140     m = B.shape[1]
141     top_left = np.kron(gamma.T, np.eye(n)) - np.kron(np.eye(q), acl)
142     top_right = -np.kron(np.eye(q), B)
143     bottom_left = np.kron(np.eye(q), ccl)
144     bottom_right = np.kron(np.eye(q), D)
145     system = np.block([
146         [top_left, top_right],
147         [bottom_left, bottom_right],
148     ])
149     rhs = np.concatenate([
150         rhs_top.reshape(-1, order="F"),
151         rhs_bottom.reshape(-1, order="F"),
152     ])
153     solution = np.linalg.solve(system, rhs)
154     p_matrix = solution[: n * q].reshape((n, q), order="F")
155     k_matrix = solution[n * q :].reshape((m, q), order="F")
156     return p_matrix, k_matrix
157
158
159 def simulate_case(kx, kg, kf):
160     z0 = np.concatenate([X0, WF0, WG0])
161
162     def rhs(_, state):
163         x = state[:3]
164         w_f = state[3:7]
165         w_g = state[7:10]
166         f_val = Y_F @ w_f
167         u_val = float((kx @ x.reshape(-1, 1))[0, 0] + (kg @ w_g.reshape(-1, 1))
168             [0, 0] + (kf @ w_f.reshape(-1, 1))[0, 0])
169         dx = A @ x + B[:, 0] * u_val + B_F @ f_val
170         return np.concatenate([dx, GAMMA_F @ w_f, GAMMA_G @ w_g])
171
172     sol = solve_ivp(
173         rhs,
174         (float(T[0]), float(T[-1])),
175         z0,
176         t_eval=T,
177         rtol=1e-8,
178         atol=1e-10,
179     )
180     x = sol.y[:3]

```

```

180     w_f = sol.y[3:7]
181     w_g = sol.y[7:10]
182     f = Y_F @ w_f
183     g = (Y_G @ w_g).reshape(-1)
184     u = np.array(
185         [
186             float((kx @ x[:, i : i + 1])[0, 0] + (kg @ w_g[:, i : i + 1])[0, 0]
187                  + (kf @ w_f[:, i : i + 1])[0, 0])
188             for i in range(x.shape[1])
189         ]
190     )
191     y = (C @ x).reshape(-1) + float(D[0, 0]) * u + (D_F @ f).reshape(-1)
192     e = g - y
193     return {
194         "t": sol.t,
195         "x": x,
196         "w_f": w_f,
197         "w_g": w_g,
198         "f": f,
199         "g": g,
200         "u": u,
201         "y": y,
202         "e": e,
203     }
204
205 def save_plot(path, t=None, legend_outside=False):
206     ax = plt.gca()
207     if t is not None:
208         ax.set_xlim(float(t[0]), float(t[-1]))
209         ax.margins(x=0)
210     plt.grid(True, alpha=0.3)
211     if legend_outside:
212         plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=2,
213                   frameon=True)
214     else:
215         plt.legend(loc="best", frameon=True)
216     plt.tight_layout()
217     plt.savefig(path, format="svg")
218     plt.close()
219
220 def plot_disturbance_reference(data, name):
221     plt.figure(figsize=(5.4, 3.35))
222     plt.plot(data["t"], data["f"][0], color=COLORS["f1"], label=r"$f_1(t)$")
223     plt.plot(data["t"], data["f"][1], color=COLORS["f2"], label=r"$f_2(t)$")

```

```

224     plt.plot(data["t"], data["g"], color=COLORS["g"], linestyle="--",
225             label=r"$g(t)$")
226     plt.xlabel("t, c")
227     plt.ylabel("Сигналы")
228     save_plot(PLOTS_DIR / f"{name}_fg.svg", t=data["t"])
229
230 def plot_state(data, name):
231     plt.figure(figsize=(5.4, 3.35))
232     plt.plot(data["t"], data["x"][0], color=COLORS["x1"], label=r"$x_1(t)$")
233     plt.plot(data["t"], data["x"][1], color=COLORS["x2"], label=r"$x_2(t)$")
234     plt.plot(data["t"], data["x"][2], color=COLORS["x3"], label=r"$x_3(t)$")
235     plt.xlabel("t, c")
236     plt.ylabel("x(t)")
237     save_plot(PLOTS_DIR / f"{name}_x.svg", t=data["t"])
238
239
240 def plot_control(data, name):
241     plt.figure(figsize=(5.4, 3.35))
242     plt.plot(data["t"], data["u"], color=COLORS["u"], label=r"$u(t)$")
243     plt.xlabel("t, c")
244     plt.ylabel("u(t)")
245     save_plot(PLOTS_DIR / f"{name}_u.svg", t=data["t"])
246
247
248 def plot_output(data, name):
249     plt.figure(figsize=(5.4, 3.35))
250     plt.plot(data["t"], data["g"], color=COLORS["g"], linewidth=2.0,
251             label=r"$g(t)$")
252     plt.plot(data["t"], data["y"], color=COLORS["y"], linestyle="--",
253             label=r"$y(t)$")
254     plt.xlabel("t, c")
255     plt.ylabel("Выход")
256     save_plot(PLOTS_DIR / f"{name}_y.svg", t=data["t"])
257
258 def plot_error(data, name):
259     plt.figure(figsize=(5.4, 3.35))
260     plt.plot(data["t"], data["e"], color=COLORS["e"], label=r"$e(t)=g(t)-$
261             y(t)$")
262     plt.xlabel("t, c")
263     plt.ylabel("e(t)")
264     save_plot(PLOTS_DIR / f"{name}_e.svg", t=data["t"])
265
266 def main():

```

```

266     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
267
268     ab = A @ B
269     a2b = A @ ab
270     ctrl = ct.ctrb(A, B)
271     eig_a = np.linalg.eigvals(A)
272     eig_gamma_f = np.linalg.eigvals(GAMMA_F)
273     eig_gamma_g = np.linalg.eigvals(GAMMA_G)
274
275     hautus_a = {}
276     for eig in eig_a:
277         h_mat = np.hstack((eig * np.eye(3) - A, B))
278         hautus_a[f"{eig.real:.6f}{eig.imag:+.6f}j"] = {
279             "lambda": {"re": float(np.real(eig)), "im": float(np.imag(eig))},
280             "H": serialize_matrix(h_mat),
281             "rank": int(np.linalg.matrix_rank(h_mat)),
282         }
283
284     k_lqr, _, _ = ct.lqr(A, B, Q_LQR, R_LQR)
285     k_feedback = -np.asarray(k_lqr, dtype=float)
286     acl = A + B @ k_feedback
287     ccl = C + D @ k_feedback
288
289     pg, kg = solve_fd(
290         GAMMA_G,
291         np.zeros((3, 3)),
292         Y_G,
293         acl,
294         ccl,
295     )
296     pf, kf = solve_fd(
297         GAMMA_F,
298         B_F @ Y_F,
299         -D_F @ Y_F,
300         acl,
301         ccl,
302     )
303
304     hautus_fd = {"tracking": {}, "disturbance": {}}
305     for eig in eig_gamma_g:
306         mat = np.block([[A - eig * np.eye(3), B], [C, D]])
307         hautus_fd["tracking"][f"{eig.real:.6f}{eig.imag:+.6f}j"] = {
308             "lambda": {"re": float(np.real(eig)), "im": float(np.imag(eig))},
309             "R": serialize_matrix(mat),
310             "rank": int(np.linalg.matrix_rank(mat)),
311         }

```

```

312     for eig in eig_gamma_f:
313         mat = np.block([[A - eig * np.eye(3), B], [C, D]])
314         hautus_fd["disturbance"][f"{eig.real:.6f}{eig.imag:+.6f}j"] = {
315             "lambda": {"re": float(np.real(eig)), "im": float(np.imag(eig))},
316             "R": serialize_matrix(mat),
317             "rank": int(np.linalg.matrix_rank(mat)),
318         }
319
320     scenarios = {}
321     for key, cfg in SCENARIOS.items():
322         kx = k_feedback if cfg["Kx"] is None else cfg["Kx"]
323         kg_use = kg if cfg["Kg"] is None else cfg["Kg"]
324         kf_use = kf if cfg["Kf"] is None else cfg["Kf"]
325         data = simulate_case(kx, kg_use, kf_use)
326         scenarios[key] = {
327             "label": cfg["label"],
328             "Kx": round_matrix(kx),
329             "Kg": round_matrix(kg_use),
330             "Kf": round_matrix(kf_use),
331             "max_u_abs": float(np.max(np.abs(data["u"]))),
332             "max_x_abs": float(np.max(np.abs(data["x"]))),
333             "max_y_abs": float(np.max(np.abs(data["y"]))),
334             "max_e_abs": float(np.max(np.abs(data["e"]))),
335             "e_final": float(data["e"][-1]),
336             "t": data["t"].tolist(),
337             "x": data["x"].tolist(),
338             "f": data["f"].tolist(),
339             "g": data["g"].tolist(),
340             "u": data["u"].tolist(),
341             "y": data["y"].tolist(),
342             "e": data["e"].tolist(),
343         }
344
345     plot_state(data, key)
346     plot_output(data, key)
347     if key == "open":
348         plot_disturbance_reference(data, key)
349     else:
350         plot_control(data, key)
351         plot_error(data, key)
352
353     payload = {
354         "A": round_matrix(A),
355         "B": round_matrix(B),
356         "B_f": round_matrix(B_F),
357         "C": round_matrix(C),

```

```

358     "D": round_matrix(D),
359     "D_f": round_matrix(D_F),
360     "Gamma_f": round_matrix(GAMMA_F),
361     "Y_f": round_matrix(Y_F),
362     "Gamma_g": round_matrix(GAMMA_G),
363     "Y_g": round_matrix(Y_G),
364     "w_f0": round_matrix(WF0),
365     "w_g0": round_matrix(WG0),
366     "AB": round_matrix(ab),
367     "A2B": round_matrix(a2b),
368     "U": round_matrix(ctrl),
369     "rank_U": int(np.linalg.matrix_rank(ctrl)),
370     "det_U": float(np.linalg.det(ctrl)),
371     "eig_A": serialize_values(np.round(eig_a, 6)),
372     "eig_Gamma_f": serialize_values(np.round(eig_gamma_f, 6)),
373     "eig_Gamma_g": serialize_values(np.round(eig_gamma_g, 6)),
374     "hautus_A": hautus_a,
375     "fd_rank_checks": hautus_fd,
376     "Q_lqr": round_matrix(Q_LQR),
377     "R_lqr": round_matrix(R_LQR),
378     "K": round_matrix(k_feedback),
379     "eig_Acl": serialize_values(np.round(np.linalg.eigvals(ac1), 6)),
380     "C_plus_DK": round_matrix(cc1),
381     "P_g": round_matrix(pg),
382     "K_g": round_matrix(kg),
383     "P_f": round_matrix(pf),
384     "K_f": round_matrix(kf),
385     "residual_tracking_top": float(np.max(np.abs(pg @ GAMMA_G - ac1 @ pg -
386     B @ kg))),
386     "residual_tracking_bottom": float(np.max(np.abs(cc1 @ pg + D @ kg -
387     Y_G))),
387     "residual_disturbance_top": float(np.max(np.abs(pf @ GAMMA_F - ac1 @ pf
388     - B @ kf - B_F @ Y_F))),
388     "residual_disturbance_bottom": float(np.max(np.abs(cc1 @ pf + D @ kf +
389     D_F @ Y_F))),
389     "scenarios": scenarios,
390 }
391 RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
392 encoding="utf-8")
393
394 if __name__ == "__main__":
395     main()

```

Код для задания 2

```
1 import json
```

 Python

```
2  import os
3  from pathlib import Path
4
5  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
6
7  import control as ct
8  import matplotlib.pyplot as plt
9  import numpy as np
10 from scipy.integrate import solve_ivp
11 from scipy.linalg import solve_sylvester
12
13 BASE_DIR = Path(__file__).resolve().parents[1]
14 PLOTS_DIR = BASE_DIR / "plots_task2"
15 RESULTS_PATH = Path(__file__).with_name("task2_results.json")
16
17 A = np.array([
18     [4.0, 1.0, 6.0],
19     [2.0, 0.0, 2.0],
20     [-2.0, -2.0, -4.0],
21 ])
22 B = np.array([[2.0], [-2.0], [2.0]])
23 B_F = np.array([
24     [-2.0, 1.0],
25     [0.0, 0.0],
26     [2.0, 0.0],
27 ])
28 C = np.array([[2.0, -2.0, 1.0]])
29 D = np.array([[10.0]])
30 D_F = np.array([[0.0, 7.0]])
31
32 GAMMA_F = np.array([
33     [-40.0, 16.0, 9.0, 7.0],
34     [-64.0, 25.0, 14.0, 12.0],
35     [-26.0, 11.0, 7.0, 3.0],
36     [-48.0, 18.0, 14.0, 8.0],
37 ])
38 Y_F = np.array([
39     [-12.0, 2.0, 2.0, 6.0],
40     [8.0, -8.0, 12.0, -3.0],
41 ])
42
43 GAMMA_G = np.array([
44     [0.0, 1.0, 0.0],
45     [-1.0, 0.0, 0.0],
46     [0.0, 0.0, 0.0],
47 ])
```

```

48 Y_G = np.array([[19.0, 0.0, -10.0]])
49
50 GAMMA_Q = np.diag([-2.0, -3.0, -4.0])
51 Y_Q = np.array([[1.0], [1.0], [1.0]])
52 OBSERVER_POLES = np.array([-2.0, -3.0, -4.0, -5.0, -6.0, -7.0, -8.0])
53
54 X0 = np.zeros(3)
55 WF0 = np.ones(4)
56 WG0 = np.array([0.0, 1.0, 1.0])
57 WBAR0 = np.zeros(3)
58 XBAR_HAT0 = np.zeros(7)
59 T = np.linspace(0.0, 6.0, 1801)
60
61 COLORS = {
62     "u": "tab:red",
63     "y": "tab:blue",
64     "g": "tab:orange",
65     "e": "tab:red",
66     "x1": "tab:blue",
67     "x2": "tab:orange",
68     "x3": "tab:green",
69     "xhat": "cyan",
70     "wf1": "tab:blue",
71     "wf2": "tab:orange",
72     "wf3": "tab:green",
73     "wf4": "tab:red",
74     "wgh1": "tab:blue",
75     "wgh2": "tab:orange",
76     "wgh3": "tab:green",
77 }
78
79
80 def round_matrix(mat, digits=6):
81     return np.round(np.asarray(mat, dtype=float), digits).tolist()
82
83
84 def serialize_values(values, digits=6):
85     out = []
86     for value in np.asarray(values).flatten():
87         if abs(np.imag(value)) < 1e-9:
88             out.append(round(float(np.real(value)), digits))
89         else:
90             out.append({
91                 "re": round(float(np.real(value)), digits),
92                 "im": round(float(np.imag(value)), digits),
93             })

```

```

94     return out
95
96
97 def solve_fd(gamma, rhs_top, rhs_bottom, acl, ccl):
98     n = A.shape[0]
99     q = gamma.shape[0]
100    top_left = np.kron(gamma.T, np.eye(n)) - np.kron(np.eye(q), acl)
101    top_right = -np.kron(np.eye(q), B)
102    bottom_left = np.kron(np.eye(q), ccl)
103    bottom_right = np.kron(np.eye(q), D)
104    system = np.block([
105        [top_left, top_right],
106        [bottom_left, bottom_right],
107    ])
108    rhs = np.concatenate([
109        rhs_top.reshape(-1, order="F"),
110        rhs_bottom.reshape(-1, order="F"),
111    ])
112    solution = np.linalg.solve(system, rhs)
113    p_matrix = solution[: n * q].reshape((n, q), order="F")
114    k_matrix = solution[n * q :].reshape((1, q), order="F")
115    return p_matrix, k_matrix
116
117
118 def save_plot(path, t=None, t_window=None, legend_outside=False,
119             legend_ncol=4):
120     ax = plt.gca()
121     if t_window is not None:
122         ax.set_xlim(float(t_window[0]), float(t_window[1]))
123         ax.margins(x=0)
124     elif t is not None:
125         ax.set_xlim(float(t[0]), float(t[-1]))
126         ax.margins(x=0)
127     plt.grid(True, alpha=0.3)
128     if legend_outside:
129         plt.legend(
130             loc="upper center",
131             bbox_to_anchor=(0.5, -0.2),
132             ncol=legend_ncol,
133             frameon=True,
134         )
135     else:
136         plt.legend(loc="best", frameon=True)
137     plt.tight_layout()
138     plt.savefig(path, format="svg")
139     plt.close()

```

```

139
140
141 def plot_control(data):
142     plt.figure(figsize=(5.4, 3.35))
143     plt.plot(data["t"], data["u"], color=COLORS["u"], label=r"$u(t)$")
144     plt.xlabel("t, c")
145     plt.ylabel("u(t)")
146     save_plot(PLOTS_DIR / "task2_u.svg", t=data["t"])
147
148
149 def plot_output(data):
150     plt.figure(figsize=(5.4, 3.35))
151     plt.plot(data["t"], data["g"], color=COLORS["g"], linewidth=2.0,
152             label=r"$g(t)$")
153     plt.plot(data["t"], data["y"], color=COLORS["y"], linestyle="--",
154             label=r"$y(t)$")
155     plt.xlabel("t, c")
156     plt.ylabel("Выход")
157     save_plot(PLOTS_DIR / "task2_y.svg", t=data["t"])
158
159
160 def plot_error(data):
161     plt.figure(figsize=(5.4, 3.35))
162     plt.plot(data["t"], data["e"], color=COLORS["e"], label=r"$e(t)=g(t)-y(t)$")
163     plt.xlabel("t, c")
164     plt.ylabel("e(t)")
165     save_plot(PLOTS_DIR / "task2_e.svg", t=data["t"])
166
167
168 def plot_wf_compare(data):
169     plt.figure(figsize=(5.7, 3.75))
170     labels = [
171         (0, r"$w_{f,1}(t)$", r"$\hat{w}_{f,1}(t)$", "tab:blue"),
172         (1, r"$w_{f,2}(t)$", r"$\hat{w}_{f,2}(t)$", "tab:orange"),
173         (2, r"$w_{f,3}(t)$", r"$\hat{w}_{f,3}(t)$", "tab:green"),
174         (3, r"$w_{f,4}(t)$", r"$\hat{w}_{f,4}(t)$", "tab:red"),
175     ]
176     for idx, true_label, est_label, color in labels:
177         plt.plot(data["t"], data["aug_true"][idx], color=color, linewidth=1.5,
178                 label=true_label)
179         plt.plot(data["t"], data["aug_hat"][idx], color=color, linestyle="--",
180                 linewidth=1.1, label=est_label)
181     plt.xlabel("t, c")
182     plt.ylabel("Компоненты")
183     save_plot(PLOTS_DIR / "task2_wf_compare.svg", t_window=(0.0, 4.0),
184             legend_outside=True, legend_ncol=4)

```

```

180
181
182 def plot_wf_error(data):
183     plt.figure(figsize=(5.7, 3.75))
184     colors = ["tab:blue", "tab:orange", "tab:green", "tab:red"]
185     names = [r"$e_{f,1}(t)$", r"$e_{f,2}(t)$", r"$e_{f,3}(t)$", r"$e_{f,4}(t)$"]
186     for idx, (name, color) in enumerate(zip(names, colors)):
187         plt.plot(data["t"], data["e_f"][idx], color=color, label=name)
188     plt.xlabel("t, c")
189     plt.ylabel(r"$e_f(t)$")
190     save_plot(PLOTS_DIR / "task2_wf_error.svg", t_window=(0.0, 4.0))
191
192
193 def plot_x_compare(data):
194     plt.figure(figsize=(5.7, 3.75))
195     labels = [
196         (4, r"$x_1(t)$", r"$\hat{x}_1(t)$", "tab:purple"),
197         (5, r"$x_2(t)$", r"$\hat{x}_2(t)$", "tab:brown"),
198         (6, r"$x_3(t)$", r"$\hat{x}_3(t)$", "tab:pink"),
199     ]
200     for idx, true_label, est_label, color in labels:
201         plt.plot(data["t"], data["aug_true"][idx], color=color, linewidth=1.5, label=true_label)
202         plt.plot(data["t"], data["aug_hat"][idx], color=color, linestyle="--", linewidth=1.1, label=est_label)
203     plt.xlabel("t, c")
204     plt.ylabel("Компоненты")
205     save_plot(PLOTS_DIR / "task2_x_compare.svg", t_window=(0.0, 4.0))
206
207
208 def plot_x_error(data):
209     plt.figure(figsize=(5.7, 3.75))
210     colors = ["tab:purple", "tab:brown", "tab:pink"]
211     names = [r"$e_{x,1}(t)$", r"$e_{x,2}(t)$", r"$e_{x,3}(t)$"]
212     for idx, (name, color) in enumerate(zip(names, colors), start=4):
213         plt.plot(data["t"], data["e_f"][idx], color=color, label=name)
214     plt.xlabel("t, c")
215     plt.ylabel(r"$e_x(t)$")
216     save_plot(PLOTS_DIR / "task2_x_error.svg", t_window=(0.0, 4.0))
217
218
219 def plot_wg_compare(data):
220     plt.figure(figsize=(5.4, 3.45))
221     colors = ["tab:blue", "tab:orange", "tab:green"]
222     for idx, color in enumerate(colors):

```

```

223     plt.plot(data["t"], data["w_g"][idx], color=color, linewidth=1.5,
               label=rf"$w_{\{g,\{idx+1\}\}}(t)$")
224     plt.plot(data["t"], data["w_g_hat"][idx], color=color, linestyle="--",
               linewidth=1.1, label=rf"$\hat{w}_{\{g,\{idx+1\}\}}(t)$")
225     plt.xlabel("t, c")
226     plt.ylabel("Компоненты")
227     save_plot(PLOTS_DIR / "task2_wg_compare.svg", t=data["t"],
               legend_outside=True, legend_ncol=3)
228
229
230 def plot_wg_error(data):
231     plt.figure(figsize=(5.4, 3.45))
232     colors = ["tab:blue", "tab:orange", "tab:green"]
233     for idx, color in enumerate(colors):
234         plt.plot(data["t"], data["e_g"][idx], color=color, label=rf"$e_{\{g,\{idx+1\}\}}(t)$")
235     plt.xlabel("t, c")
236     plt.ylabel(r"$e_g(t)$")
237     save_plot(PLOTS_DIR / "task2_eg.svg", t=data["t"])
238
239
240 def main():
241     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
242
243     k_lqr, _, _ = ct.lqr(A, B, np.eye(3), np.array([[1.0]]))
244     k_feedback = -np.asarray(k_lqr, dtype=float)
245     acl = A + B @ k_feedback
246     ccl = C + D @ k_feedback
247     _, k_g = solve_fd(GAMMA_G, np.zeros((3, 3)), Y_G, acl, ccl)
248     _, k_f = solve_fd(GAMMA_F, B_F @ Y_F, -D_F @ Y_F, acl, ccl)
249
250     q_matrix = solve_sylvester(-GAMMA_Q, GAMMA_G, Y_Q @ Y_G)
251     q_inv = np.linalg.inv(q_matrix)
252
253     a_ext = np.block([
254         [GAMMA_F, np.zeros((4, 3))],
255         [B_F @ Y_F, A],
256     ])
257     b_ext = np.vstack([np.zeros((4, 1)), B])
258     c_ext = np.hstack([D_F @ Y_F, C])
259     observability_rank = int(np.linalg.matrix_rank(ct.obsv(a_ext, c_ext)))
260
261     l_matrix = -ct.place(a_ext.T, c_ext.T, OBSERVER_POLES).T
262
263     z0 = np.concatenate([X0, WF0, WG0, WBAR0, XBAR_HAT0])
264

```

```

265     def rhs(_, state):
266         x = state[:3]
267         w_f = state[3:7]
268         w_g = state[7:10]
269         w_bar = state[10:13]
270         x_ext_hat = state[13:20]
271
272         f_val = Y_F @ w_f
273         g_val = float((Y_G @ w_g.reshape(-1, 1))[0, 0])
274
275         w_f_hat = x_ext_hat[:4]
276         x_hat = x_ext_hat[4:]
277         w_g_hat = q_inv @ w_bar
278
279         u_val = float(
280             (k_feedback @ x_hat.reshape(-1, 1))[0, 0]
281             + (k_g @ w_g_hat.reshape(-1, 1))[0, 0]
282             + (k_f @ w_f_hat.reshape(-1, 1))[0, 0]
283         )
284         y_val = float((C @ x.reshape(-1, 1))[0, 0] + D[0, 0] * u_val + (D_F @
285             f_val.reshape(-1, 1))[0, 0])
286         y_hat = float((c_ext @ x_ext_hat.reshape(-1, 1))[0, 0])
287
288         dx = A @ x + B[:, 0] * u_val + B_F @ f_val
289         dw_f = GAMMA_F @ w_f
290         dw_g = GAMMA_G @ w_g
291         dw_bar = GAMMA_Q @ w_bar + Y_Q[:, 0] * g_val
292         dx_ext_hat = a_ext @ x_ext_hat + b_ext[:, 0] * u_val + l_matrix[:, 0] *
293             (y_hat + D[0, 0] * u_val - y_val)
294         return np.concatenate([dx, dw_f, dw_g, dw_bar, dx_ext_hat])
295
296     sol = solve_ivp(
297         rhs,
298         (float(T[0]), float(T[-1])),
299         z0,
300         t_eval=T,
301         rtol=1e-8,
302         atol=1e-10,
303     )
304
305     x = sol.y[:3]
306     w_f = sol.y[3:7]
307     w_g = sol.y[7:10]
308     w_bar = sol.y[10:13]
309     x_ext_hat = sol.y[13:20]
310

```

```

309     w_f_hat = x_ext_hat[:4]
310     x_hat = x_ext_hat[4:]
311     w_g_hat = q_inv @ w_bar
312
313     f = Y_F @ w_f
314     g = (Y_G @ w_g).reshape(-1)
315     u = np.array(
316         [
317             float(
318                 (k_feedback @ x_hat[:, i : i + 1])[0, 0]
319                 + (k_g @ w_g_hat[:, i : i + 1])[0, 0]
320                 + (k_f @ w_f_hat[:, i : i + 1])[0, 0]
321             )
322             for i in range(T.size)
323         ]
324     )
325     y = (C @ x).reshape(-1) + D[0, 0] * u + (D_F @ f).reshape(-1)
326     aug_true = np.vstack([w_f, x])
327     aug_hat = np.vstack([w_f_hat, x_hat])
328     e_f = aug_true - aug_hat
329     e_g = w_g - w_g_hat
330     e = g - y
331
332     data = {
333         "t": sol.t,
334         "u": u,
335         "y": y,
336         "g": g,
337         "w_g": w_g,
338         "w_g_hat": w_g_hat,
339         "aug_true": aug_true,
340         "aug_hat": aug_hat,
341         "e_f": e_f,
342         "e_g": e_g,
343         "e": e,
344     }
345
346     plot_control(data)
347     plot_wf_compare(data)
348     plot_wf_error(data)
349     plot_x_compare(data)
350     plot_x_error(data)
351     plot_wg_compare(data)
352     plot_wg_error(data)
353     plot_output(data)
354     plot_error(data)

```

```

355
356     payload = {
357         "A": round_matrix(A),
358         "B": round_matrix(B),
359         "B_f": round_matrix(B_F),
360         "C": round_matrix(C),
361         "D": round_matrix(D),
362         "D_f": round_matrix(D_F),
363         "Gamma_f": round_matrix(GAMMA_F),
364         "Y_f": round_matrix(Y_F),
365         "Gamma_g": round_matrix(GAMMA_G),
366         "Y_g": round_matrix(Y_G),
367         "Gamma_q": round_matrix(GAMMA_Q),
368         "Y_q": round_matrix(Y_Q),
369         "Q": round_matrix(q_matrix),
370         "Q_det": float(np.linalg.det(q_matrix)),
371         "Q_cond": float(np.linalg.cond(q_matrix)),
372         "Q_residual": float(np.max(np.abs(q_matrix @ GAMMA_G - GAMMA_Q @
q_matrix - Y_Q @ Y_G))),
373         "K": round_matrix(k_feedback),
374         "K_g": round_matrix(k_g),
375         "K_f": round_matrix(k_f),
376         "A_ext": round_matrix(a_ext),
377         "B_ext": round_matrix(b_ext),
378         "C_ext": round_matrix(c_ext),
379         "observability_rank": observability_rank,
380         "observer_poles": round_matrix(OBSERVER_POLES),
381         "L": round_matrix(l_matrix),
382         "eig_error_wg": serialize_values(np.round(np.linalg.eigvals(GAMMA_Q),
6)),
383         "eig_error_ext": serialize_values(np.round(np.linalg.eigvals(a_ext +
l_matrix @ c_ext), 6)),
384         "max_u_abs": float(np.max(np.abs(u))),
385         "max_x_abs": float(np.max(np.abs(x))),
386         "max_ef_abs": float(np.max(np.abs(e_f))),
387         "max_eg_abs": float(np.max(np.abs(e_g))),
388         "e_final": float(e[-1]),
389         "e_last_10pct": float(np.max(np.abs(e[int(0.9 * len(e)) :]))),
390         "t": sol.t.tolist(),
391         "u": u.tolist(),
392         "y": y.tolist(),
393         "g": g.tolist(),
394         "w_g": w_g.tolist(),
395         "w_g_hat": w_g_hat.tolist(),
396         "aug_true": aug_true.tolist(),
397         "aug_hat": aug_hat.tolist(),

```

```

398     "e_f": e_f.tolist(),
399     "e_g": e_g.tolist(),
400     "e": e.tolist(),
401 }
402 RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
403                           encoding="utf-8")
404
405 if __name__ == "__main__":
406     main()

```

Код для задания 3

```

1  import json
2  import os
3  from pathlib import Path
4
5  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
6
7  import control as ct
8  import matplotlib.pyplot as plt
9  import numpy as np
10 from scipy.integrate import solve_ivp
11 from scipy.linalg import solve_sylvester
12
13 BASE_DIR = Path(__file__).resolve().parents[1]
14 PLOTS_DIR = BASE_DIR / "plots_task3"
15 RESULTS_PATH = Path(__file__).with_name("task3_results.json")
16
17 A = np.array([
18     [4.0, 1.0, 6.0],
19     [2.0, 0.0, 2.0],
20     [-2.0, -2.0, -4.0],
21 ])
22 B = np.array([[2.0], [-2.0], [2.0]])
23 B_F = np.array([
24     [-2.0, 1.0],
25     [0.0, 0.0],
26     [2.0, 0.0],
27 ])
28 C = np.array([[2.0, -2.0, 1.0]])
29 D = np.array([[10.0]])
30 D_F = np.array([[0.0, 7.0]])
31
32 GAMMA_F = np.array([
33     [-40.0, 16.0, 9.0, 7.0],

```

```

34     [-64.0, 25.0, 14.0, 12.0],
35     [-26.0, 11.0, 7.0, 3.0],
36     [-48.0, 18.0, 14.0, 8.0],
37 ]
38 Y_F = np.array([
39     [-12.0, 2.0, 2.0, 6.0],
40     [8.0, -8.0, 12.0, -3.0],
41 ])
42
43 GAMMA_G = np.array([
44     [0.0, 1.0, 0.0],
45     [-1.0, 0.0, 0.0],
46     [0.0, 0.0, 0.0],
47 ])
48 Y_G = np.array([[19.0, 0.0, -10.0]])
49
50 GAMMA_Q = np.diag([-2.0, -3.0, -4.0])
51 Y_Q = np.array([[1.0], [1.0], [1.0]])
52
53 STATE_OBSERVER_POLES = np.array([-2.0, -3.0, -4.0, -5.0])
54 OUTPUT_OBSERVER_GAMMA = np.diag([-2.0, -3.0, -4.0, -5.0])
55 OUTPUT_OBSERVER_Y = np.ones((4, 1))
56
57 X0 = np.zeros(3)
58 WF0 = np.ones(4)
59 WG0 = np.array([0.0, 1.0, 1.0])
60 WGBAR0 = np.zeros(3)
61 OBS0 = np.zeros(4)
62 T = np.linspace(0.0, 8.0, 2401)
63
64
65 def round_matrix(mat, digits=6):
66     return np.round(np.asarray(mat, dtype=float), digits).tolist()
67
68
69 def serialize_values(values, digits=6):
70     out = []
71     for value in np.asarray(values).flatten():
72         if abs(np.imag(value)) < 1e-9:
73             out.append(round(float(np.real(value)), digits))
74         else:
75             out.append({
76                 "re": round(float(np.real(value)), digits),
77                 "im": round(float(np.imag(value)), digits),
78             })
79     return out

```

```

80
81
82 def solve_fd(gamma, rhs_top, rhs_bottom, acl, ccl):
83     n = A.shape[0]
84     q = gamma.shape[0]
85     top_left = np.kron(gamma.T, np.eye(n)) - np.kron(np.eye(q), acl)
86     top_right = -np.kron(np.eye(q), B)
87     bottom_left = np.kron(np.eye(q), ccl)
88     bottom_right = np.kron(np.eye(q), D)
89     system = np.block([
90         [top_left, top_right],
91         [bottom_left, bottom_right],
92     ])
93     rhs = np.concatenate([
94         rhs_top.reshape(-1, order="F"),
95         rhs_bottom.reshape(-1, order="F"),
96     ])
97     solution = np.linalg.solve(system, rhs)
98     p_matrix = solution[: n * q].reshape((n, q), order="F")
99     k_matrix = solution[n * q :].reshape((1, q), order="F")
100     return p_matrix, k_matrix
101
102
103 def save_plot(path, t=None, legend_outside=False, legend_ncol=4):
104     ax = plt.gca()
105     if t is not None:
106         ax.set_xlim(float(t[0]), float(t[-1]))
107         ax.margins(x=0)
108     plt.grid(True, alpha=0.3)
109     if legend_outside:
110         plt.legend(
111             loc="upper center",
112             bbox_to_anchor=(0.5, -0.2),
113             ncol=legend_ncol,
114             frameon=True,
115         )
116     else:
117         plt.legend(loc="best", frameon=True)
118     plt.tight_layout()
119     plt.savefig(path, format="svg")
120     plt.close()
121
122
123 def plot_control(data, prefix):
124     plt.figure(figsize=(5.4, 3.35))
125     plt.plot(data["t"], data["u"], color="tab:red", label=r"$u(t)$")

```

```

126     plt.xlabel("t, c")
127     plt.ylabel("u(t)")
128     save_plot(PLOTS_DIR / f"{prefix}_u.svg", t=data["t"])
129
130
131 def plot_state(data, prefix):
132     plt.figure(figsize=(5.4, 3.35))
133     plt.plot(data["t"], data["x"][0], color="tab:blue", label=r"$x_1(t)$")
134     plt.plot(data["t"], data["x"][1], color="tab:orange", label=r"$x_2(t)$")
135     plt.plot(data["t"], data["x"][2], color="tab:green", label=r"$x_3(t)$")
136     plt.xlabel("t, c")
137     plt.ylabel("x(t)")
138     save_plot(PLOTS_DIR / f"{prefix}_x.svg", t=data["t"])
139
140
141 def plot_wf_compare(data, prefix):
142     plt.figure(figsize=(5.6, 3.65))
143     colors = ["tab:blue", "tab:orange", "tab:green", "tab:red"]
144     for idx, color in enumerate(colors):
145         plt.plot(data["t"], data["w_f"][idx], color=color, linewidth=1.5,
146                 label=r"$w_{\{f,\{idx+1\}}(t)$")
147         plt.plot(data["t"], data["w_f_hat"][idx], color=color, linestyle="--",
148                 linewidth=1.1, label=r"$\hat{w}_{\{f,\{idx+1\}}(t)$")
149     plt.xlabel("t, c")
150     plt.ylabel("Компоненты")
151     save_plot(PLOTS_DIR / f"{prefix}_wf_compare.svg", t=data["t"],
152             legend_outside=True, legend_ncol=4)
153
154
155 def plot_ef(data, prefix):
156     plt.figure(figsize=(5.6, 3.65))
157     colors = ["tab:blue", "tab:orange", "tab:green", "tab:red"]
158     for idx, color in enumerate(colors):
159         plt.plot(data["t"], data["e_f"][idx], color=color, label=r"$e_{\{f,\{idx+1\}}(t)$")
160     plt.xlabel("t, c")
161     plt.ylabel(r"$e_f(t)$")
162     save_plot(PLOTS_DIR / f"{prefix}_ef.svg", t=data["t"])
163
164
165 def plot_output(data, prefix):
166     plt.figure(figsize=(5.4, 3.35))
167     plt.plot(data["t"], data["g"], color="tab:orange", linewidth=2.0,
168             label=r"$g(t)$")
169     plt.plot(data["t"], data["y"], color="tab:blue", linestyle="--",
170             label=r"$y(t)$")
171     plt.xlabel("t, c")

```

```

167     plt.ylabel("Выход")
168     save_plot(PLOTS_DIR / f"{prefix}_y.svg", t=data["t"])
169
170
171 def plot_error(data, prefix):
172     plt.figure(figsize=(5.4, 3.35))
173     plt.plot(data["t"], data["e"], color="tab:red", label=r"$e(t)=g(t)-y(t)$")
174     plt.xlabel("t, c")
175     plt.ylabel("e(t)")
176     save_plot(PLOTS_DIR / f"{prefix}_e.svg", t=data["t"])
177
178
179 def main():
180     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
181
182     k_lqr, _, _ = ct.lqr(A, B, np.eye(3), np.array([[1.0]]))
183     k_feedback = -np.asarray(k_lqr, dtype=float)
184     acl = A + B @ k_feedback
185     ccl = C + D @ k_feedback
186     _, k_g = solve_fd(GAMMA_G, np.zeros((3, 3)), Y_G, acl, ccl)
187     _, k_f = solve_fd(GAMMA_F, B_F @ Y_F, -D_F @ Y_F, acl, ccl)
188
189     q_wg = solve_sylvester(-GAMMA_Q, GAMMA_G, Y_Q @ Y_G)
190     q_wg_inv = np.linalg.inv(q_wg)
191
192     c_bar = np.linalg.inv(B_F.T @ B_F) @ B_F.T
193     l_state = ct.place(GAMMA_F.T, Y_F.T, STATE_OBSERVER_POLES).T
194     f_state = GAMMA_F - l_state @ Y_F
195
196     d_f_y_f = D_F @ Y_F
197     q_output = solve_sylvester(-OUTPUT_OBSERVER_GAMMA, GAMMA_F,
198                                OUTPUT_OBSERVER_Y @ d_f_y_f)
199     q_output_inv = np.linalg.inv(q_output)
200
201     observers = {}
202     for mode in ("state", "output"):
203         z0 = np.concatenate([X0, WF0, WG0, WGBAR0, OBS0])
204
205         def rhs(_, state):
206             x = state[:3]
207             w_f = state[3:7]
208             w_g = state[7:10]
209             w_g_bar = state[10:13]
210             obs = state[13:17]
211
212             f_val = Y_F @ w_f

```

```

212         g_val = float((Y_G @ w_g.reshape(-1, 1))[0, 0])
213         w_g_hat = q_wg_inv @ w_g_bar
214
215         if mode == "state":
216             w_f_hat = obs + l_state @ (c_bar @ x)
217         else:
218             w_f_hat = q_output_inv @ obs
219
220         u_val = float(
221             (k_feedback @ x.reshape(-1, 1))[0, 0]
222             + (k_g @ w_g_hat.reshape(-1, 1))[0, 0]
223             + (k_f @ w_f_hat.reshape(-1, 1))[0, 0]
224         )
225         y_val = float((C @ x.reshape(-1, 1))[0, 0] + D[0, 0] * u_val + (D_F
226             @ f_val.reshape(-1, 1))[0, 0])
227
228         dx = A @ x + B[:, 0] * u_val + B_F @ f_val
229         dw_f = GAMMA_F @ w_f
230         dw_g = GAMMA_G @ w_g
231         dw_g_bar = GAMMA_Q @ w_g_bar + Y_Q[:, 0] * g_val
232
233         if mode == "state":
234             dobs = f_state @ obs + (f_state @ l_state @ c_bar - l_state @
235                 c_bar @ A) @ x - (l_state @ c_bar @ B[:, 0] * u_val
236             else:
237                 f_meas = y_val - float((C @ x.reshape(-1, 1))[0, 0]) - D[0, 0]
238                 * u_val
239                 dobs = OUTPUT_OBSERVER_GAMMA @ obs + OUTPUT_OBSERVER_Y[:, 0] *
240                     f_meas
241
242         return np.concatenate([dx, dw_f, dw_g, dw_g_bar, dobs])
243
244     sol = solve_ivp(
245         rhs,
246         (float(T[0]), float(T[-1])),
247         z0,
248         t_eval=T,
249         rtol=1e-8,
250         atol=1e-10,
251     )
252
253     x = sol.y[:3]
254     w_f = sol.y[3:7]
255     w_g = sol.y[7:10]
256     w_g_bar = sol.y[10:13]
257     obs = sol.y[13:17]

```

```

254     w_g_hat = q_wg_inv @ w_g_bar
255     if mode == "state":
256         w_f_hat = obs + l_state @ (c_bar @ x)
257     else:
258         w_f_hat = q_output_inv @ obs
259
260     f = Y_F @ w_f
261     g = (Y_G @ w_g).reshape(-1)
262     u = np.array(
263         [
264             float(
265                 (k_feedback @ x[:, i : i + 1])[0, 0]
266                 + (k_g @ w_g_hat[:, i : i + 1])[0, 0]
267                 + (k_f @ w_f_hat[:, i : i + 1])[0, 0]
268             )
269             for i in range(T.size)
270         ]
271     )
272     y = (C @ x).reshape(-1) + D[0, 0] * u + (D_F @ f).reshape(-1)
273     e_f = w_f - w_f_hat
274     e_g = w_g - w_g_hat
275     e = g - y
276
277     data = {
278         "t": sol.t,
279         "x": x,
280         "w_f": w_f,
281         "w_f_hat": w_f_hat,
282         "w_g": w_g,
283         "w_g_hat": w_g_hat,
284         "u": u,
285         "y": y,
286         "g": g,
287         "e_f": e_f,
288         "e_g": e_g,
289         "e": e,
290     }
291     prefix = "task3_state" if mode == "state" else "task3_output"
292     plot_control(data, prefix)
293     plot_state(data, prefix)
294     plot_wf_compare(data, prefix)
295     plot_ef(data, prefix)
296     plot_output(data, prefix)
297     plot_error(data, prefix)
298
299     observers[mode] = {

```

```

300         "max_u_abs": float(np.max(np.abs(u))),
301         "max_x_abs": float(np.max(np.abs(x))),
302         "max_ef_abs": float(np.max(np.abs(e_f))),
303         "max_eg_abs": float(np.max(np.abs(e_g))),
304         "e_final": float(e[-1]),
305         "e_last_10pct": float(np.max(np.abs(e[int(0.9 * len(e)) :]))),
306         "t": sol.t.tolist(),
307         "x": x.tolist(),
308         "w_f": w_f.tolist(),
309         "w_f_hat": w_f_hat.tolist(),
310         "w_g": w_g.tolist(),
311         "w_g_hat": w_g_hat.tolist(),
312         "u": u.tolist(),
313         "y": y.tolist(),
314         "g": g.tolist(),
315         "e_f": e_f.tolist(),
316         "e_g": e_g.tolist(),
317         "e": e.tolist(),
318     }
319
320     payload = {
321         "A": round_matrix(A),
322         "B": round_matrix(B),
323         "B_f": round_matrix(B_F),
324         "C": round_matrix(C),
325         "D": round_matrix(D),
326         "D_f": round_matrix(D_F),
327         "Gamma_f": round_matrix(GAMMA_F),
328         "Y_f": round_matrix(Y_F),
329         "Gamma_g": round_matrix(GAMMA_G),
330         "Y_g": round_matrix(Y_G),
331         "K": round_matrix(k_feedback),
332         "K_g": round_matrix(k_g),
333         "K_f": round_matrix(k_f),
334         "Gamma_q": round_matrix(GAMMA_Q),
335         "Y_q": round_matrix(Y_Q),
336         "Q_wg": round_matrix(q_wg),
337         "state_observer": {
338             "C_bar": round_matrix(c_bar),
339             "L": round_matrix(l_state),
340             "F": round_matrix(f_state),
341             "eig_F": serialize_values(np.round(np.linalg.eigvals(f_state), 6)),
342             **observers["state"],
343         },
344         "output_observer": {
345             "Gamma": round_matrix(OUTPUT_OBSERVER_GAMMA),

```

```
346         "Y": round_matrix(OUTPUT_OBSERVER_Y),
347         "Q": round_matrix(q_output),
348         "Q_det": float(np.linalg.det(q_output)),
349         "Q_cond": float(np.linalg.cond(q_output)),
350         "Q_residual": float(np.max(np.abs(q_output @ GAMMA_F -
351                                     OUTPUT_OBSERVER_GAMMA @ q_output - OUTPUT_OBSERVER_Y @ d_f_y_f))),
352         **observers["output"],
353     },
354     RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
355                               encoding="utf-8")
356
357 if __name__ == "__main__":
358     main()
```